

UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Felipe Echeverria Vilhalva

**Arquitetura de Análise de Redes Parlamentares Baseada em Teoria dos Grafos para
Identificação de Estruturas de Influência**

DOURADOS – MS

2026

Felipe Echeverria Vilhalva

**Arquitetura de Análise de Redes Parlamentares Baseada em Teoria dos Grafos para
Identificação de Estruturas de Influência**

Projeto Final de Curso, apresentado à Universidade Estadual de Mato Grosso do Sul (UEMS), Bacharelado em Ciência da Computação, Dourados, para obtenção do título de Bacharel em Bacharelado em Ciência da Computação.

Orientador: Prof. Dr. Rubens Barbosa Filho

DOURADOS – MS

2026

Felipe Echeverria Vilhalva

**Arquitetura de Análise de Redes Parlamentares Baseada em Teoria dos Grafos para
Identificação de Estruturas de Influência**

Banca Examinadora:

Prof. Dr. Rubens Barbosa Filho (Orientador)
Instituição: Universidade Estadual de Mato Grosso do Sul
Área: Computação

Prof. Dr. Nome Completo do Membro da Banca
Instituição: Universidade Estadual de Mato Grosso do Sul
Área: Computação

Prof. Dr. Nome Completo do Membro da Banca
Instituição: Universidade Estadual de Mato Grosso do Sul
Área: Computação

*Aos meus pais, que me deram a oportunidade
de chegar até aqui, e ao meu orientador,
que tornou este trabalho possível.*

AGRADECIMENTOS

Agradeço, primeiramente, à minha família — em especial aos meus pais — pelo apoio incondicional, pelos sacrifícios silenciosos e pela base que tornou toda esta caminhada possível.

Ao meu orientador, Prof. Dr. Rubens Barbosa Filho, pela paciência, pelas discussões enriquecedoras e pela orientação rigorosa que moldou este trabalho. Sua disposição em desafiar ideias e em apontar caminhos foi determinante para o resultado final.

Aos professores do curso de Bacharelado em Ciência da Computação da Universidade Estadual de Mato Grosso do Sul (UEMS) — Unidade Universitária de Dourados, por todo o conhecimento transmitido ao longo da graduação e pela formação técnica e humana que carrego comigo.

Aos meus colegas de curso, com quem dividi madrugadas de estudo, ansiedades de prova, descobertas e amizades que pretendo levar para a vida.

Aos meus amigos, que entenderam as ausências e celebraram cada conquista deste percurso como se fosse delas também.

“Este é um lugar para estar. Isto é tudo o que você tem, mas ainda é alguma coisa. Ruas e luzes de sódio. O céu, o mundo. Você ainda está vivo.”

— Volição, *Disco Elysium*

RESUMO

Este trabalho aborda a lacuna entre a abundância de dados legislativos abertos da Câmara dos Deputados do Brasil e a escassez de ferramentas analíticas reprodutíveis capazes de extrair, desses dados, evidências mensuráveis sobre estruturas de influência política. Sob o paradigma de *Design Science Research*, projetou-se, implementou-se e validou-se uma arquitetura de software modular e auditável que transforma registros de coautoria legislativa do período 2022–2025 em uma rede analisável por Teoria dos Grafos. A rede é modelada como projeção unimodal ponderada de um grafo bipartido, adotando o peso $1/(n_p - 1)$ e um filtro de massa de signatários (`max_authors=30`) para mitigar a inflação de arestas. Sobre a rede resultante, calculam-se quatro métricas de centralidade e detectam-se comunidades por dois algoritmos independentes — Louvain e Label Propagation —, cuja significância é avaliada por teste de permutação contra modelo nulo ponderado. A hipótese central, de que a estrutura de comunidades é não-aleatória, foi corroborada nos quatro anos analisados ($p < 0,005$), com modularidade observada entre 0,596 e 0,640; as comunidades detectadas recuperam a organização partidária no nível de coalizão (*Adjusted Rand Index* $\approx 0,30$); a articulação legislativa concentra-se em poucos parlamentares; e a centralidade de coautoria revela-se medida de articulação, não de poder institucional formal. A contribuição central, coerente com o paradigma adotado, é o próprio artefato: uma ferramenta reprodutível, coberta por testes automatizados e empacotada em contêiner, disponibilizada em código aberto para auditoria, extensão e reuso.

Palavras-Chave: ciência de redes; ciência social computacional; engenharia de software; teoria dos grafos; detecção de comunidades; redes parlamentares.

ABSTRACT

This work addresses the gap between the abundance of open legislative data from the Brazilian Chamber of Deputies and the scarcity of reproducible analytical tools able to extract, from such data, measurable evidence about structures of political influence. Under the *Design Science Research* paradigm, a modular and auditable software architecture was designed, implemented and validated to transform legislative co-authorship records from the 2022–2025 period into a network analyzable through Graph Theory. The network is modeled as a weighted one-mode projection of a bipartite graph, adopting the weight $1/(n_p - 1)$ and a signatory-mass filter (`max_authors= 30`) to mitigate edge inflation. Over the resulting network, four centrality metrics are computed and communities are detected by two independent algorithms — Louvain and Label Propagation —, whose significance is assessed by a permutation test against a weighted null model. The central hypothesis, that the community structure is non-random, was corroborated in all four analyzed years ($p < 0.005$), with observed modularity between 0.596 and 0.640; the detected communities recover the partisan organization at the coalition level (*Adjusted Rand Index* ≈ 0.30); legislative articulation is concentrated in a few members; and co-authorship centrality proves to be a measure of articulation, not of formal institutional power. The central contribution, consistent with the adopted paradigm, is the artifact itself: a reproducible tool, covered by automated tests and packaged in a container, released as open source for auditing, extension and reuse.

Keywords: network science; computational social science; software engineering; graph theory; community detection; parliamentary networks.

LISTA DE ILUSTRAÇÕES

Figura 1	Diagrama de Arquitetura Limpa	20
Figura 2	Representação de um grafo desconexo não direcionado.	22
Figura 3	Representações de grafos bipartidos	23
Figura 4	Representação de um grafo aplicado ao contexto	24
Figura 5	Framework de Pesquisa em Design Science	31
Figura 6	Grafo bipartido de coautoria: deputados (D) e proposições (P), com n_p indicando o número de coautores de cada matéria.	35
Figura 7	Projeção unimodal do grafo bipartido da Figura 6 sobre o conjunto D , com os pesos calculados pela Equação 12.	36
Figura 8	Efeito da heurística de ponderação na projeção unimodal.	37
Figura 9	Diagrama de componentes e fluxo de dados do <i>pipeline</i>	41
Figura 10	Fluxo de junção e filtragem dos arquivos CSV até a construção da rede de coautoria.	44
Figura 11	Curvas de Lorenz da concentração das centralidades em 2025: quanto mais a curva se afasta da diagonal de igualdade, maior a concentração da métrica em poucos parlamentares.	55
Figura 12	Legenda de cores por comunidade (Louvain) da Figura 13. A legenda de cores por partido foi omitida em razão do período eleitoral.	58
Figura 13	Rede de coautoria de 2025 (mesmo <i>layout</i> ForceAtlas 2), colorida por comunidade detectada (a) e por filiação partidária (b). Arestas com peso baixo omitidas para legibilidade.	59
Figura 14	Evolução das métricas estruturais (2022–2025): modularidade, ARI, densidade e concentração (Gini do autovetor).	60

LISTA DE TABELAS

Tabela 1 – Métricas estruturais da rede de coautoria por ano.	52
Tabela 2 – Efeito do filtro de massa de signatários sobre a densidade (2025).	53
Tabela 3 – Sensibilidade da rede ao limiar <code>max_authors</code> (2025).	53
Tabela 4 – Deputados de maior centralidade em 2025 (identidades anonimizadas). . . .	54
Tabela 5 – Concentração da centralidade em 2025 (coeficiente de Gini e participação dos 10 mais centrais).	55
Tabela 6 – Concordância entre Louvain e Label Propagation (ARI) por ano.	56
Tabela 7 – Teste de significância da estrutura comunitária (modelo nulo ponderado, 200 permutações).	57
Tabela 8 – Composição partidária das principais comunidades (Louvain, 2025; partidos anonimizados).	57

LISTA DE ABREVIATURAS E SIGLAS

ARI	Adjusted Rand Index
CSV	Comma-Separated Values
DSR	Design Science Research
EMC	Emenda
GEXF	Graph Exchange XML Format
H1	Hipótese de Pesquisa (estrutura não-aleatória)
I/O	Input/Output
PDL	Projeto de Decreto Legislativo
PEC	Proposta de Emenda à Constituição
PFC	Projeto Final de Curso
PL	Projeto de Lei
PLP	Projeto de Lei Complementar
SOLID	Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion
TCC	Trabalho de Conclusão de Curso
TDD	Test-Driven Development
UEMS	Universidade Estadual de Mato Grosso do Sul

LISTA DE SÍMBOLOS

G	Grafo
B	Grafo bipartido
V	Conjunto de vértices
E	Conjunto de arestas
D	Conjunto de deputados
P	Conjunto de proposições
w	Função de peso da aresta
Q	Modularidade
ΔQ	Variação de modularidade
w_{ij}	Peso da aresta entre os vértices i e j
n_p	Número de coautores da proposição p
k_i	Grau do vértice i
s_i	Centralidade de grau (força) do vértice i
$C_B(v)$	Centralidade de intermediação do vértice v
$C_C(v)$	Centralidade de proximidade do vértice v
$C_E(v)$	Centralidade de autovetor do vértice v
$C_x(t)$	Rótulo comunitário do vértice x na iteração t
α	Nível de significância estatística

SUMÁRIO

1	INTRODUÇÃO	15
1.1	PROBLEMA DE PESQUISA	15
1.2	Objetivos	16
1.2.1	Objetivo Geral	16
1.2.2	Objetivos Específicos	16
1.3	HIPÓTESE	16
1.4	ESTRUTURA DO TRABALHO	17
2	REVISÃO DA LITERATURA	19
2.1	ENGENHARIA DE SOFTWARE E ARQUITETURA DE SISTEMAS . . .	19
2.1.1	Arquitetura Limpa (Clean Architecture)	19
2.1.2	Princípios SOLID	20
2.1.3	Camadas do Projeto e Reprodutibilidade	21
2.2	TEORIA DOS GRAFOS E CIÊNCIA DE REDES	21
2.2.1	Fundamentos de Redes Complexas e Grafos	21
2.2.2	Grafos Bipartidos e Projeção Unimodal	22
2.2.3	Modelagem de Redes Sociais e Coautoria	23
2.2.4	Métricas de Centralidade e Influência	24
2.3	DETECÇÃO DE COMUNIDADES (AGRUPAMENTOS PARLAMENTARES)	26
2.3.1	Modularidade (Q)	26
2.3.2	Algoritmo de Louvain	27
2.3.3	Algoritmo Label Propagation	28
2.4	TRABALHOS CORRELATOS	28
3	METODOLOGIA	30
3.1	CARACTERIZAÇÃO DA PESQUISA	30
3.1.1	O Método DSR (Design Science Research)	30
3.1.2	Fases do Método	31
3.1.3	Uso de Ferramentas de Inteligência Artificial	32
3.2	PROCEDIMENTOS DE COLETA E TRATAMENTO DE DADOS	32
3.2.1	Fonte dos Dados	32
3.2.2	Recorte Temporal	33
3.2.3	Ferramental de Processamento	33
3.3	ESTRATÉGIA DE MODELAGEM E MITIGAÇÃO DE VIÉS	34
3.3.1	Definição Topológica	34
3.3.2	Heurística de Ponderação e Normalização do Grafo	36

3.4	SELEÇÃO DE ALGORITMOS E MÉTRICAS ANALÍTICAS	38
3.4.1	Métricas de Centralidade	38
3.4.2	Deteccção de Comunidades	39
3.5	MEDIDA DE ROBUSTEZ: ADJUSTED RAND INDEX (ARI)	39
3.6	AVALIAÇÃO DO ARTEFATO (PROCEDIMENTOS DE VALIDAÇÃO) . .	39
3.6.1	Validação Qualitativa	40
3.6.2	Validação Quantitativa e Reprodutibilidade	40
4	DESENVOLVIMENTO	41
4.1	DEFINIÇÃO DA ARQUITETURA DO SISTEMA E ESTRUTURA MODU- LAR	41
4.1.1	Models (Entidades)	42
4.1.2	Core (Casos de Uso)	42
4.1.3	Extraction e Processing (Adaptadores de Interface)	43
4.1.4	Repository (Frameworks e Drivers)	44
4.2	MODELAGEM DO GRAFO PARLAMENTAR E HEURÍSTICA DE PON- DERAÇÃO	46
4.2.1	Tipos de Proposição Considerados	46
4.3	IMPLEMENTAÇÃO DO NÚCLEO ANALÍTICO COMPUTACIONAL . .	47
4.3.1	Cálculo de Centralidades	47
4.3.2	Deteccção de Comunidades	47
4.3.3	Persistência dos Resultados Analíticos	48
4.4	QUALIDADE, INFRAESTRUTURA E REPRODUTIBILIDADE	48
4.4.1	Suíte de testes	48
4.4.2	Empacotamento Docker	49
4.4.3	Orquestração do projeto	49
4.4.4	Repositório aberto	49
4.4.5	Integração Contínua e Controle de Versão	49
5	RESULTADOS E DISCUSSÕES	51
5.1	VALIDAÇÃO DA ARQUITETURA E REPRODUTIBILIDADE	51
5.1.1	Desempenho Computacional	51
5.1.2	Geração de Artefatos (Interoperabilidade)	51
5.1.3	Avaliação da Arquitetura: Desacoplamento e Extensibilidade	51
5.2	ANÁLISE DE CENTRALIDADE E INFLUÊNCIA PARLAMENTAR . . .	52
5.2.1	O Grafo Resultante	52
5.2.2	Os Deputados mais Influentes	54
5.3	DETECCÃO DE COMUNIDADES E TESTE DA HIPÓTESE CENTRAL .	56
5.3.1	Comparação Louvain vs. Label Propagation	56
5.3.2	Análise pela Modularidade (Q) — Teste da Hipótese H1	57

5.4	ANÁLISE TEMPORAL (2022–2025)	60
5.4.1	Estabilidade do Padrão Estrutural	60
5.4.2	Discussão Longitudinal	61
6	CONCLUSÃO	62
6.1	SÍNTESE DOS ACHADOS	62
6.2	CONTRIBUIÇÕES	62
6.2.1	Contribuição técnica	62
6.2.2	Contribuição analítica	63
6.3	LIMITAÇÕES	63
6.4	TRABALHOS FUTUROS	64
6.5	CONSIDERAÇÕES FINAIS	64
	REFERÊNCIAS	65
	APÊNDICE A – ESTRUTURA DO REPOSITÓRIO DE CÓDIGO . .	67

1 INTRODUÇÃO

A Câmara dos Deputados do Brasil produz, anualmente, dezenas de milhares de proposições legislativas — projetos de lei, emendas constitucionais e demais matérias — cuja autoria e coautoria são registradas e disponibilizadas publicamente pelo Portal de Dados Abertos da instituição. Esse volume massivo de registros relacionais representa, em princípio, uma fonte rica de evidências sobre a articulação política interna do legislativo. No entanto, dados brutos não se convertem automaticamente em compreensão: a passagem de centenas de milhares de registros de coautoria para afirmações fundamentadas sobre estruturas de influência exige instrumentos analíticos formais, reproduzíveis e teoricamente ancorados.

A Teoria dos Grafos oferece precisamente esse instrumental. Ao modelar a coautoria legislativa como uma rede ponderada, na qual deputados são vértices e colaborações em proposições são arestas, torna-se possível aplicar métricas consolidadas de centralidade, algoritmos de detecção de comunidades e testes estatísticos de significância. Essa abordagem, inaugurada no contexto legislativo por Fowler (2006) no estudo do Congresso dos Estados Unidos, transforma assinaturas em documentos oficiais em um objeto matemático tratável, do qual se podem extrair evidências mensuráveis sobre poder informal, coalizões e mediação política.

O presente trabalho parte da premissa de que a coautoria parlamentar não é fenômeno aleatório, mas reflexo de alinhamentos políticos, ideológicos e estratégicos. Sob essa premissa — e na esteira de Fowler (2006) —, a rede de coautoria é adotada como *proxy* mensurável de articulação política, passível de análise quantitativa. A cobertura estende-se deliberadamente de 2022 a 2025 — abrangendo a transição entre a 56^a e a 57^a Legislaturas —, permitindo tanto a análise da estrutura em regime estável quanto o estudo da sua persistência ao longo do tempo.

Por *estruturas de influência* entende-se, neste trabalho, a organização estrutural da rede de coautoria em dois níveis complementares: as *comunidades* — agrupamentos de deputados com colaboração densa, que revelam blocos e coalizões — e as *posições de centralidade* — deputados que atuam como pontes entre grupos ou como polos de articulação. O primeiro nível descreve *como* a articulação se organiza; o segundo, *quem* ocupa as posições estruturalmente mais influentes. A identificação dessas estruturas, em ambos os níveis, é o objeto de análise do artefato desenvolvido.

1.1 PROBLEMA DE PESQUISA

A despeito da disponibilidade pública dos dados e da maturidade dos métodos de análise de redes, a literatura brasileira sobre o tema apresenta ainda lacunas significativas: (i) a maioria dos estudos existentes concentra-se em períodos isolados ou em legislaturas anteriores, sem análise sistemática multi-anual recente; (ii) raros trabalhos disponibilizam código-fonte reproduzível, dificultando a verificação e a extensão dos achados; (iii) há carência de ferramentas computacio-

nais reutilizáveis que separem com clareza a engenharia de processamento dos dados e a análise política propriamente dita.

Este trabalho aborda essas lacunas por meio da construção de uma *arquitetura modular, reprodutível e auditável* para análise de redes parlamentares, fundamentada em Teoria dos Grafos e Engenharia de Software contemporânea, aplicada aos quatro anos legislativos completos do período 2022–2025.

1.2 OBJETIVOS

Esta seção apresenta o objetivo geral do projeto e os objetivos específicos que dele decorrem.

1.2.1 Objetivo Geral

Projetar, implementar e validar uma arquitetura de software reprodutível para análise de redes parlamentares da Câmara dos Deputados do Brasil, fundamentada na Teoria dos Grafos, capaz de transformar dados legislativos brutos em evidências analíticas sobre estruturas de influência política.

1.2.2 Objetivos Específicos

- a) Modelar matematicamente a rede de coautoria legislativa como projeção unimodal de um grafo bipartido, com fórmula de ponderação justificada teoricamente;
- b) Implementar *pipeline* computacional em camadas independentes, seguindo princípios de Arquitetura Limpa e SOLID, com cobertura por testes automatizados;
- c) Aplicar métricas de centralidade (grau, intermediação, proximidade, autovetor) e algoritmos de detecção de comunidades (Louvain e Label Propagation) sobre os dados dos anos 2022 a 2025;
- d) Validar estatisticamente a significância da estrutura comunitária por meio de teste de permutação contra modelo nulo preservando a sequência de graus;
- e) Discutir os achados à luz da literatura sobre comportamento legislativo brasileiro, testando a hipótese central sobre a estrutura da rede.

1.3 HIPÓTESE

Este trabalho é orientado por uma *única hipótese falsificável*, formulada em termos estritamente estruturais (topológicos) — a partir de uma propriedade mensurável do grafo, a modularidade — e sem recurso a qualquer classificação ideológica atribuída aos partidos. O procedimento que a operacionaliza e a testa é descrito no Capítulo 3, e o veredito correspondente — corroboração ou refutação — é apresentado no Capítulo 5.

- **H1 — Estrutura de comunidades não-aleatória:** a modularidade Q da rede de coautoria parlamentar é significativamente superior à obtida em grafos nulos que preservam a mesma sequência de graus (*configuration model*). Como predição falsificável, H1 é **refutada** caso a modularidade observada não se distinga estatisticamente da distribuição nula ($p \geq \alpha$) — situação em que a fragmentação em comunidades seria indistinguível de um arranjo aleatório de colaborações e a coautoria não refletiria organização política estruturada.

Coerentemente com o paradigma de *Design Science Research* adotado (Capítulo 3), H1 cumpre o papel de *condição de validade* da análise, e não de tese conclusiva sobre o sistema político: antes de interpretar centralidades e comunidades como estruturas de influência, é necessário demonstrar que a organização modular da rede não é mero artefato do acaso ou da distribuição de graus. A contribuição central do trabalho é, portanto, a arquitetura reprodutível que *viabiliza* essa análise; o teste de H1 e os resultados descritivos são a demonstração de que o artefato opera corretamente sobre o período 2022–2025.

Além do teste dessa hipótese, o trabalho produz, como resultados descritivos complementares, o cálculo das quatro métricas de centralidade para cada deputado e a comparação entre dois algoritmos independentes de detecção de comunidades — elementos que compõem a caracterização das estruturas de influência da rede e reforçam a robustez do achado central.

Essa caracterização é organizada em torno de três *perguntas norteadoras*, de natureza descritiva e exploratória — e não hipóteses a serem testadas estatisticamente:

- (i) há concentração da articulação legislativa em poucos parlamentares?
- (ii) as comunidades detectadas correspondem a partidos ou coalizões reais?
- (iii) a centralidade de coautoria prediz influência política real?

Diferentemente de H1, essas questões não são submetidas a teste de significância: são respondidas pela caracterização empírica da rede no Capítulo 5. A hipótese H1 é, ainda assim, a *condição que as viabiliza* — só faz sentido perguntar se as comunidades correspondem a partidos, por exemplo, depois de demonstrado que a estrutura comunitária não é fruto do acaso.

1.4 ESTRUTURA DO TRABALHO

O restante deste trabalho está organizado da seguinte forma. O Capítulo 2 apresenta a revisão da literatura, cobrindo os fundamentos de Engenharia de Software, Teoria dos Grafos, Ciência de Redes e Detecção de Comunidades. O Capítulo 3 descreve a metodologia, incluindo o paradigma DSR, os procedimentos de coleta e tratamento de dados, a estratégia de modelagem, a medida de robustez das partições e os procedimentos de validação. O Capítulo 4 detalha o desenvolvimento do artefato de software, com ênfase na arquitetura modular e nas decisões de implementação. O Capítulo 5 apresenta os resultados obtidos para os quatro anos analisados, com a caracterização

das centralidades e o teste da hipótese central. O Capítulo 6 conclui o trabalho com síntese dos resultados, discussão das limitações e proposição de trabalhos futuros.

2 REVISÃO DA LITERATURA

Este capítulo apresenta o arcabouço teórico que fundamenta o trabalho, organizado em três eixos complementares. A primeira seção discute os princípios de Engenharia de Software que orientam a construção do artefato, com ênfase em Arquitetura Limpa e nos princípios SOLID. A segunda seção apresenta os fundamentos matemáticos e conceituais da Teoria dos Grafos e da Ciência de Redes, incluindo as métricas de centralidade aplicadas ao longo da análise. A terceira seção aborda a detecção de comunidades em redes complexas, incluindo a métrica de modularidade e os algoritmos selecionados para este trabalho.

2.1 ENGENHARIA DE SOFTWARE E ARQUITETURA DE SISTEMAS

A construção de um software analítico voltado para a ciência de redes requer uma fundação arquitetural sólida. O objetivo primário da arquitetura de software é minimizar os recursos humanos necessários para construir e manter um determinado sistema (Martin, 2017). Para que o sistema possua uma vida longa e tolerância a mudanças, é imperativo que suas regras de negócio sejam isoladas de tecnologias externas voláteis. Nas subseções a seguir, são apresentados os conceitos e princípios de engenharia de software que fundamentam a estruturação deste trabalho.

2.1.1 Arquitetura Limpa (Clean Architecture)

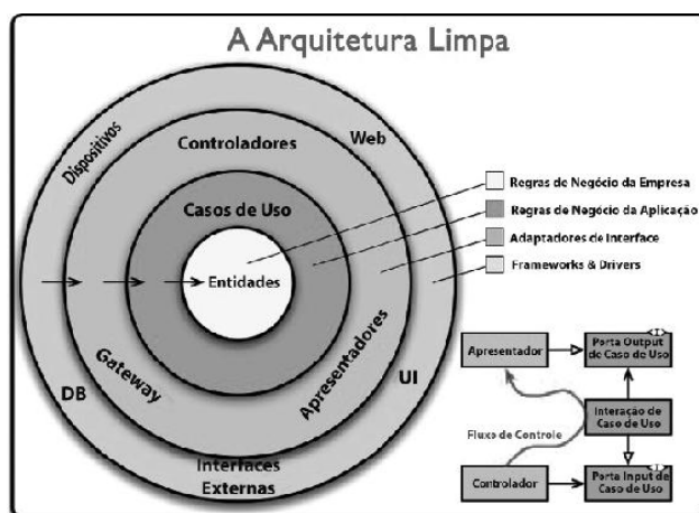
A Arquitetura Limpa, proposta por Robert C. Martin, é um modelo arquitetural que visa a separação de preocupações através da divisão do software em camadas bem definidas. O modelo produz sistemas que são independentes de framework, independentes de interface de usuário (UI), independentes de banco de dados e intrinsecamente testáveis.

O princípio fundamental que rege este modelo é a Regra de Dependência, a qual determina que as dependências de código-fonte devem apontar apenas para dentro, na direção das políticas de nível mais alto. Em uma Arquitetura Limpa clássica, o sistema é tipicamente dividido em quatro círculos concêntricos:

- a) **Entidades:** concentram as regras cruciais de negócios e os dados de domínio, sendo os elementos mais gerais e de nível mais alto do sistema;
- b) **Casos de Uso:** contêm as regras de negócio específicas da aplicação, orquestrando o fluxo de dados para e a partir das entidades;
- c) **Adaptadores de Interface:** convertem os dados do formato conveniente para as entidades e casos de uso em um formato adequado para agentes externos, como a web ou o banco de dados;

- d) **Frameworks e Drivers:** o círculo mais externo, composto por detalhes de baixo nível, como bibliotecas de banco de dados e frameworks web. Sob a ótica arquitetural, o banco de dados e a web são considerados meros detalhes ou mecanismos de entrada/saída que não devem poluir a arquitetura central do sistema.

Figura 1 – Diagrama de Arquitetura Limpa



Fonte: Arquitetura Limpa (2017, Figura 22.1).

2.1.2 Princípios SOLID

A base para a criação de componentes de nível médio bem projetados, tolerantes a mudanças e fáceis de compreender é o conjunto de princípios SOLID (Martin, 2017). Estes princípios orientam a organização de funções e estruturas de dados em classes e módulos isolados:

- SRP (Princípio da Responsabilidade Única):** estabelece que um módulo deve ser responsável por um, e apenas um, ator;
- OCP (Princípio Aberto/Fechado):** um artefato de software deve ser aberto para extensão, mas fechado para modificação;
- LSP (Princípio de Substituição de Liskov):** para que sistemas sejam construídos a partir de partes intercambiáveis, essas partes devem aderir a um contrato que permita substituição sem alterar o funcionamento do programa;
- ISP (Princípio da Segregação de Interface):** orienta os projetistas a não dependerem de coisas que não utilizam;
- DIP (Princípio da Inversão de Dependência):** o código que implementa políticas de alto nível não deve depender de detalhes de baixo nível. As dependências de código-fonte devem referir-se apenas a abstrações.

2.1.3 Camadas do Projeto e Reprodutibilidade

Neste projeto de análise de redes parlamentares, os princípios da Arquitetura Limpa e SOLID foram aplicados na definição da topologia dos diretórios. O código-fonte é estritamente isolado dos artefatos de dados gerados (*data/*), estruturando-se nas seguintes camadas modulares:

- a) **Models:** representa a camada de Entidades, abrigando classes cruciais e independentes como *Deputy*, *Proposition* e a estrutura central da rede;
- b) **Core:** atua como a camada de Casos de Uso, orquestrando a lógica pesada de grafos e centralizando algoritmos matemáticos e de detecção de comunidades, isolados de preocupações de entrada e saída (I/O);
- c) **Extraction, Processing e Visualization:** atuam como Adaptadores de Interface, consumindo dados brutos do *dataset* da Câmara e processando visualizações gráficas de maneira desacoplada das regras de negócio;
- d) **Repository:** camada que abstrai os Drivers, responsável pela persistência do banco de dados local e exportação de artefatos (*.csv*, *.gexf* e *.json*).

2.2 TEORIA DOS GRAFOS E CIÊNCIA DE REDES

A modelagem de sistemas complexos, como o ecossistema legislativo, exige uma base matemática rigorosa capaz de representar as interações entre os seus atores. Essa base é fornecida pela Teoria dos Grafos, que atua como o alicerce estrutural para a Ciência de Redes. Nas subseções seguintes, são definidos os conceitos topológicos, a modelagem de interações sociais e as métricas analíticas que viabilizam a extração de inteligência a partir de dados relacionais brutos (Newman, 2010; Diestel, 2017).

2.2.1 Fundamentos de Redes Complexas e Grafos

Na sua forma mais elementar, uma rede (ou grafo, na literatura matemática) é um catálogo dos componentes de um sistema e das interações diretas entre eles. Conforme Barabási (2016) e Diestel (2017), esses componentes são frequentemente chamados de nós (ou vértices) e as interações são chamadas de ligações (ou arestas), conforme a formulação matemática a seguir:

$$G = (V, E, w) \quad (1)$$

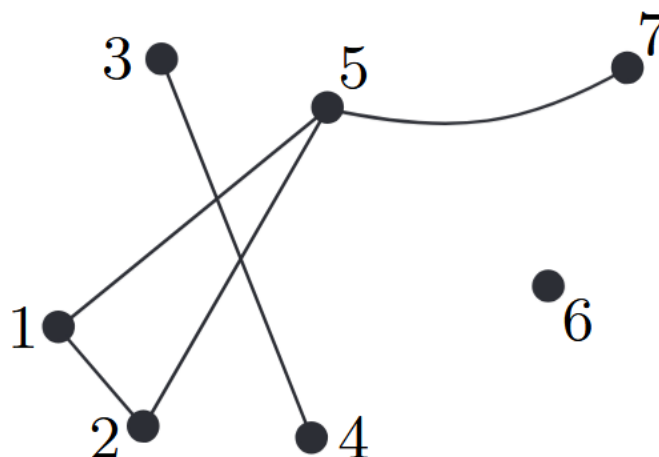
Onde:

- V é o conjunto de vértices (nós);
- $E \subseteq V \times V$ é o conjunto de arestas;

- $w : E \rightarrow \mathbb{R}$ é a função de peso que associa um valor real a cada aresta.

A representação de um grafo visualmente:

Figura 2 – Representação de um grafo desconexo não direcionado.



Fonte: Diestel (2017, p. 2).

A topologia de uma rede é ditada pela natureza de suas arestas. [Barabási \(2016\)](#) explica que os grafos podem ser direcionados, quando a relação possui um sentido único (como um link na World Wide Web), ou não direcionados, quando a relação é mútua e de via dupla (como uma linha de transmissão de energia). Além da direção, as redes do mundo real frequentemente possuem conexões com intensidades diferentes. Para modelar essas intensidades, utilizam-se os grafos ponderados, onde cada aresta recebe um peso numérico.

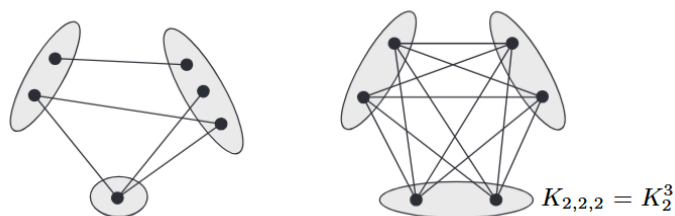
Sob a ótica da Ciência da Computação, a eficiência no processamento dessas redes depende intrinsecamente de como o grafo é alocado na memória. [Cormen et al. \(2009\)](#) detalha que existem dois modos padrões para a representação computacional: através de uma coleção de listas de adjacências ou através de uma matriz de adjacências. Enquanto a matriz consome um espaço de memória quadrático em relação aos vértices, a representação por lista de adjacências é o método preferido por oferecer uma estrutura muito mais compacta para grafos esparsos — aqueles em que o número de arestas é muito menor que o máximo possível.

2.2.2 Grafos Bipartidos e Projeção Unimodal

Uma classe particular de grafos ocupa posição central na modelagem adotada neste trabalho: os grafos bipartidos. Um grafo é dito bipartido quando seu conjunto de vértices pode ser particionado em dois subconjuntos disjuntos U e V , tais que $U \cap V = \emptyset$, de modo que toda aresta conecta necessariamente um vértice de U a um vértice de V , nunca dois vértices pertencentes ao mesmo subconjunto ([Diestel, 2017](#)). Na literatura de Ciência de Redes, essa estrutura é frequentemente denominada rede bimodal (*two-mode network*) ou rede de afiliação (*affiliation*

network), sendo o exemplo canônico a relação entre um conjunto de pessoas e um conjunto de grupos ou eventos aos quais elas pertencem (Newman, 2010), conforme o exemplo de um grafo bipartido abaixo:

Figura 3 – Representações de grafos bipartidos



Fonte: Diestel (2017, p. 15).

Como grafos bipartidos representam dois tipos distintos de entidade simultaneamente, sua análise direta por métricas de centralidade convencionais torna-se pouco intuitiva. A técnica usual consiste em derivar, a partir do grafo bipartido, uma rede unimodal (*one-mode*) por meio de uma operação de *projeção*. Na projeção sobre o subconjunto U , dois vértices de U passam a ser conectados sempre que compartilham ao menos um vizinho comum em V (Newman, 2010). Transpondo essa definição para o objeto deste trabalho, a projeção sobre o conjunto de deputados conecta dois parlamentares sempre que ambos assinam uma mesma proposição legislativa, produzindo a rede de coautoria efetivamente analisada.

A operação de projeção, contudo, introduz uma distorção estrutural conhecida. Cada elemento do subconjunto projetado que agrega n vértices do outro lado gera, na projeção, um subgrafo completo (clique) de $\binom{n}{2}$ arestas. Em consequência, grupos numerosos — proposições com muitos coautores, no caso legislativo — produzem, sozinhos, um volume desproporcional de arestas, dominando a topologia resultante e suprimindo o sinal das colaborações menores e mais específicas. Esse fenômeno de inflação de arestas é precisamente a motivação teórica para a heurística de ponderação e para os filtros de massa de signatários formalizados no Capítulo 3.

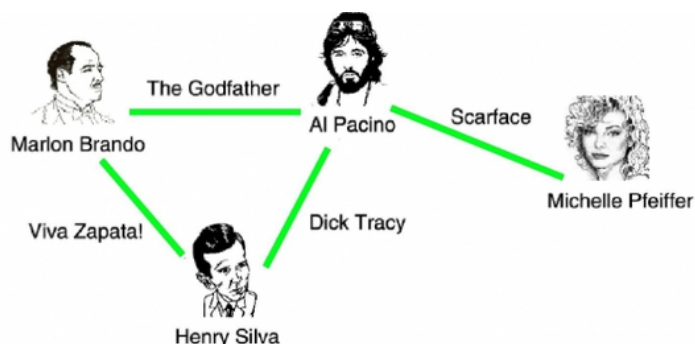
O tratamento clássico para essa distorção foi proposto por Newman (2001) no estudo de redes de colaboração científica, ao ponderar cada colaboração de forma inversamente proporcional ao número de coautores do trabalho compartilhado. A fórmula de ponderação adotada neste trabalho constitui uma adaptação direta desse princípio ao contexto parlamentar, cuja formalização completa — incluindo os filtros qualitativos derivados — é apresentada na Seção de modelagem do Capítulo 3.

2.2.3 Modelagem de Redes Sociais e Coautoria

Ao aplicar a Teoria dos Grafos à sociologia, originam-se as chamadas redes sociais. Segundo Newman (2010), nas redes sociais os vértices representam pessoas (ou grupos de pessoas), denominados atores, e as arestas representam alguma forma de interação social entre eles,

comumente chamadas de laços (*ties*), conforme a figura a seguir, que representa uma rede de atores de Hollywood, na qual dois atores estão conectados se atuaram no mesmo filme:

Figura 4 – Representação de um grafo aplicado ao contexto



Fonte: Barabási (2016).

Uma classe especialmente útil de redes sociais abordada por Newman são as redes de avaliação, onde os atores são conectados através da participação conjunta em grupos ou eventos. Um exemplo clássico e direto dessa estrutura é a rede de coautoria. Na academia, por exemplo, um ator é um autor científico e o laço é formado quando dois autores assinam um mesmo artigo.

No contexto da Câmara dos Deputados abordado neste projeto, essa exata estrutura de coautoria é adotada para a modelagem do ecossistema legislativo: os parlamentares assumem o papel de vértices e as proposições legislativas (como Projetos de Lei e PECs) assinadas em conjunto formam as arestas ponderadas da rede. Através dessa modelagem, é possível transformar assinaturas em documentos sociais em um grafo tangível de articulação política.

2.2.4 Métricas de Centralidade e Influência

Com a topologia da rede computacionalmente definida, a Ciência de Redes atua quantificando a importância estrutural de cada vértice. Newman (2010) aponta que grande parte da pesquisa em redes busca responder à pergunta: quais são os vértices mais importantes ou centrais em uma rede? Para responder a isso, existem diversas métricas matemáticas que capturam diferentes perspectivas do que significa ter influência. Neste trabalho, destacam-se quatro métricas principais:

- a) **Centralidade de Grau (*Degree*)**: é a medida mais intuitiva de centralidade, definida como o número de conexões diretas que um indivíduo possui. A premissa analítica é direta: atores com mais conexões diretas possuem maior acesso à informação e maior influência imediata no seu círculo. Em grafos ponderados (frequentemente denominada força ou *strength* do nó), o grau é calculado como o somatório dos pesos das arestas incidentes naquele vértice:

$$s_i = \sum_{j \in N(i)} w(i, j) \quad (2)$$

onde s_i representa a soma dos pesos das arestas incidentes no vértice i , e $N(i)$ representa o conjunto de vizinhos diretos de i .

- b) **Centralidade de Intermediação (*Betweenness*)**: desenvolvida inicialmente pelo sociólogo Linton Freeman (1977), esta métrica avalia a frequência com que um vértice atua como uma ponte ao longo dos caminhos mínimos (geodésicos) entre outros pares de vértices na rede. Matematicamente, ela é a razão entre a quantidade de caminhos mínimos que passam pelo vértice em questão e o número total de caminhos mínimos que conectam cada par de nós:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (3)$$

onde $C_B(v)$ é a fração de caminhos mínimos entre todos os pares que passam por v (Freeman, 1977).

- c) **Centralidade de Proximidade (*Closeness*)**: esta métrica avalia a distância geodésica média de um vértice em relação a todos os outros vértices acessíveis na rede. Nós com alta proximidade conseguem disseminar ideias ou influenciar toda a comunidade de maneira mais ágil:

$$C_C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)} \quad (4)$$

onde n é o número de vértices e $d(v, u)$ é a distância geodésica entre v e u ; ou seja, $C_C(v)$ é o inverso da distância geodésica *média* do vértice v aos demais vértices da rede.

- d) **Centralidade de Autovetor (*Eigenvector*)**: uma extensão natural do grau, a centralidade de autovetor pondera a qualidade das relações (Bonacich, 1972). O modelo matemático propõe que a importância de um vértice é proporcional à soma das importâncias dos seus vizinhos, incorporando o peso das conexões na versão ponderada:

$$C_E(v) = \frac{1}{\lambda} \sum_{u \in N(v)} w(v, u) C_E(u) \quad (5)$$

em que λ é o maior autovalor da matriz de adjacências ponderada da rede (Bonacich, 1987).

2.3 DETECÇÃO DE COMUNIDADES (AGRUPAMENTOS PARLAMENTARES)

A detecção de comunidades é o processo de encontrar divisões naturais em uma rede, buscando grupos de vértices onde existem muitas arestas internamente e poucas arestas conectando grupos distintos. Segundo [Newman \(2010\)](#), diferentemente do particionamento de grafos tradicional (onde o número e o tamanho dos grupos são rigidamente predefinidos), a detecção de comunidades é uma ferramenta exploratória que busca descobrir a estrutura de larga escala de forma autônoma. No contexto deste projeto, as comunidades extraídas do grafo representam os agrupamentos parlamentares, revelando blocos partidários, facções informais ou fortes alinhamentos ideológicos no legislativo.

2.3.1 Modularidade (Q)

A modularidade é a métrica formal mais consolidada para avaliar a qualidade de uma partição em comunidades em uma rede. Proposta originalmente por [Newman & Girvan \(2004\)](#) e formalizada matematicamente em [Newman \(2010\)](#), a modularidade Q mede a diferença entre a fração de arestas que conectam vértices da mesma comunidade no grafo observado e a fração esperada em um grafo aleatório com a mesma sequência de graus. Sua formulação matemática é:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (6)$$

Onde A_{ij} é a matriz de adjacências (igual ao peso da aresta w_{ij} em grafos ponderados), k_i é o grau do vértice i , m é o número total de arestas, c_i é a comunidade à qual o vértice i pertence, e $\delta(c_i, c_j)$ é a função delta de Kronecker, igual a 1 se $c_i = c_j$ e 0 caso contrário.

Valores de Q variam tipicamente entre $-0,5$ e 1 . Conforme [Newman \(2010\)](#), valores acima de $0,3$ indicam estrutura comunitária significativa em redes reais; valores próximos de zero indicam que a partição é estatisticamente indistinguível da aleatória; e valores negativos indicam anti-comunidades. A modularidade fornece, portanto, um critério objetivo para comparar partições alternativas geradas por diferentes algoritmos.

Um aspecto central da formulação da modularidade merece destaque, pois fundamenta o procedimento de validação estatística adotado neste trabalho. O termo $k_i k_j / 2m$ presente na Equação 6 não é arbitrário: ele representa o peso ou intensidade esperada da conexão entre os vértices i e j em um *modelo nulo*, isto é, em um grafo aleatório que preserva a sequência de graus da rede original mas redistribui as conexões ao acaso ([Newman, 2010](#)). A modularidade mede, portanto, exatamente o excesso de arestas internas às comunidades em relação ao que se esperaria por puro acaso, dada a mesma distribuição de graus.

Esse modelo nulo, conhecido na literatura como *configuration model*, é a referência estatística contra a qual a significância de uma estrutura comunitária pode ser avaliada. Uma modularidade elevada só constitui evidência robusta de organização não-aleatória quando supera de forma clara os valores obtidos sobre realizações desse modelo nulo. É precisamente essa comparação —

entre a modularidade observada e a distribuição de modularidades de grafos nulos com igual sequência de graus — que o Capítulo 3 formaliza como teste de permutação, e cujos resultados são reportados no Capítulo 5.

2.3.2 Algoritmo de Louvain

O algoritmo de Louvain, proposto por [Blondel et al. \(2008\)](#), é um método heurístico de otimização gulosa que busca maximizar a modularidade Q por meio de duas fases iterativas. Na primeira fase, cada vértice é inicialmente atribuído à sua própria comunidade. Em seguida, para cada vértice i , considera-se a remoção de i de sua comunidade atual e sua inserção em cada uma das comunidades de seus vizinhos, escolhendo o movimento que produz o maior ganho positivo em Q . A fase termina quando nenhum movimento produz ganho. Na segunda fase, as comunidades resultantes são contraídas em supernodos e o processo recomeça. As iterações continuam até que nenhuma mudança aumente a modularidade.

Sua popularidade decorre de três propriedades práticas: (i) complexidade computacional aproximadamente linear no número de arestas para grafos esparsos, viabilizando aplicação a redes com milhares de nós; (ii) qualidade comparável a algoritmos exatos em benchmarks como o de [Lancichinetti, Fortunato & Radicchi \(2008\)](#); e (iii) implementações maduras na biblioteca NetworkX, garantindo reprodutibilidade.

Há, contudo, uma limitação conhecida: o algoritmo é sensível à ordem de processamento dos vértices, podendo produzir partições ligeiramente diferentes em execuções distintas. No presente trabalho, esse efeito foi controlado pela fixação de uma semente aleatória (*seed*) constante.

No artigo original do Louvain, [Blondel et al. \(2008\)](#) apresentam a fórmula exata usada na primeira fase do algoritmo. Ela calcula, com baixíssimo custo computacional, qual seria a variação de modularidade (ΔQ), ou ganho de modularidade, ao mover um nó isolado i para dentro de uma comunidade vizinha C :

$$\Delta Q = \left[\frac{\Sigma_{in} + 2k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right] \quad (7)$$

onde Σ_{in} é a soma dos pesos das arestas internas à comunidade C , Σ_{tot} é a soma dos pesos de todas as arestas incidentes em vértices de C , k_i é o grau ponderado do vértice i , $k_{i,in}$ é a soma dos pesos das arestas de i para vértices já pertencentes a C , e m é a soma dos pesos de todas as arestas da rede.

No livro *Networks*, [Newman \(2010\)](#) demonstra que, para fins de particionamento de grafos e otimização espectral, a modularidade pode ser reescrita elegantemente em notação matricial:

$$Q = \frac{1}{2m} \mathbf{s}^\top \mathbf{B} \mathbf{s} \quad (8)$$

onde $\mathbf{s}$ é o vetor de atribuição de comunidades e $\mathbf{B}$ é a chamada matriz de modularidade, definida por $B_{ij} = A_{ij} - k_i k_j / 2m$.

2.3.3 Algoritmo Label Propagation

O algoritmo Label Propagation, proposto por [Raghavan, Albert & Kumara \(2007\)](#), parte de uma abordagem distinta: em vez de otimizar uma função objetivo global, ele propaga rótulos locais entre vértices vizinhos até a convergência por meio de um processo de busca por consenso. Inicialmente, cada vértice recebe um rótulo único. Na formulação clássica síncrona do algoritmo, o rótulo de um nó x no passo iterativo t é determinado estritamente a partir dos rótulos de seus vizinhos no passo anterior $t - 1$, conforme a Equação 9:

$$C_x(t) = f(C_{x_1}(t-1), C_{x_2}(t-1), \dots, C_{x_k}(t-1)) \quad (9)$$

onde $x_1, \dots, x_k$ são os vizinhos diretos de x e a função f retorna o rótulo de maior frequência (moda) entre os vizinhos, resolvendo empates de forma uniformemente aleatória. No entanto, estruturas bipartidas ou em formato de estrela causam oscilações infinitas de rótulos sob essa dinâmica síncrona. Para contornar essa limitação estrutural, os autores propõem uma dinâmica de atualização assíncrona, expressa na Equação 10:

$$C_x(t) = f(C_{x_{i_1}}(t), \dots, C_{x_{i_m}}(t), C_{x_{i_{m+1}}}(t-1), \dots, C_{x_{i_k}}(t-1)) \quad (10)$$

onde os vizinhos indexados de i_1 a i_m já foram atualizados no passo corrente t , enquanto os vizinhos de i_{m+1} a i_k mantêm os rótulos do passo anterior $t - 1$.

Seja $d_i^{C_j}$ a soma dos pesos das arestas do nó i conectadas a vizinhos com o rótulo C_j . O processo iterativo de propagação assíncrona encerra-se estritamente quando, para cada nó i que carrega um rótulo C_m , a Equação 11 é satisfeita para todos os rótulos alternativos j ativos na rede:

$$d_i^{C_m} \geq d_i^{C_j}, \quad \forall j \quad (11)$$

Isso garante estabilidade local, uma vez que nenhum vértice individualmente terá incentivo para alterar seu agrupamento comunitário.

2.4 TRABALHOS CORRELATOS

A aplicação de análise de redes ao comportamento legislativo não é inédita, e situar o presente trabalho nesse cenário é essencial para delimitar sua contribuição. Esta seção revisa, de forma sintética, os principais antecedentes internacionais e nacionais que orientam a modelagem aqui adotada.

No cenário internacional, o marco fundador da análise de redes de coautoria legislativa é o trabalho de [Fowler \(2006\)](#), que modelou a rede de coautoria (*cosponsorship*) do Congresso dos Estados Unidos e demonstrou que a posição estrutural de um parlamentar na rede — e não apenas sua filiação partidária — é preditora de influência legislativa. Esse estudo estabeleceu a coautoria como *proxy* mensurável de articulação política e consolidou o uso de métricas de

centralidade como indicadores de poder informal — método adotado neste trabalho, ainda que aplicado a um contexto institucional distinto.

No cenário brasileiro, a literatura sobre comportamento legislativo tradicionalmente privilegia a análise de votações nominais como indicador de coesão e disciplina partidária. [Figueiredo & Limongi \(1998\)](#) constituem referência central ao evidenciar que o presidencialismo de coalizão brasileiro produz padrões de comportamento agregado por blocos, enquanto [Zucco \(2009\)](#) discute a heterogeneidade interna dos partidos e os limites da filiação como preditor de alinhamento. Este trabalho dialoga com essa tradição, mas dela se distingue ao adotar a coautoria — e não a votação — como substrato relacional, e ao priorizar a construção de um artefato de software reprodutível.

É justamente nesse cruzamento que reside a lacuna endereçada: são raros, no contexto brasileiro, os estudos que combinam análise formal de redes de coautoria, cobertura multi-anual recente e disponibilização de código-fonte auditável. O presente trabalho posiciona-se nessa interseção, aproximando o instrumental consolidado por [Fowler \(2006\)](#) da realidade da Câmara dos Deputados do Brasil, com ênfase em reprodutibilidade computacional.

3 METODOLOGIA

Este capítulo descreve o paradigma de pesquisa adotado, os procedimentos de coleta e tratamento dos dados, a estratégia de modelagem do grafo parlamentar, a seleção dos algoritmos e métricas analíticas, a medida de robustez das partições e os procedimentos de validação aplicados ao artefato construído.

3.1 CARACTERIZAÇÃO DA PESQUISA

A natureza desta pesquisa é caracterizada como aplicada, com abordagem quantitativa e finalidade exploratório-descritiva. Aplicada, pois objetiva a construção de um artefato de software com propósito prático imediato. Quantitativa, pois fundamenta-se em métricas matematicamente formalizadas e em teste estatístico de significância. Exploratório-descritiva, pois pretende não apenas descrever a estrutura da rede legislativa, mas também explorar relações que ainda não foram sistematicamente quantificadas para o período 2022–2025.

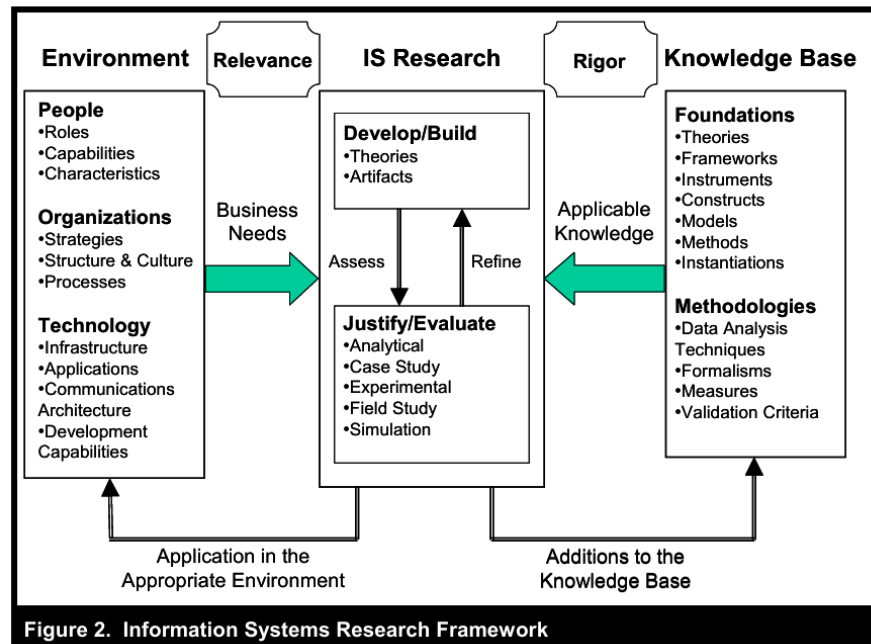
3.1.1 O Método DSR (Design Science Research)

O paradigma metodológico adotado neste trabalho é o *Design Science Research* (DSR), conforme formalizado por [Hevner et al. \(2004\)](#) e [Wieringa \(2014\)](#). Diferentemente das ciências naturais, que explicam fenômenos existentes, e das ciências sociais empíricas, que descrevem comportamentos observáveis, o DSR é orientado à *produção de artefatos* (constructos, modelos, métodos ou instanciações de software) projetados para solucionar problemas práticos identificados em um contexto real.

Como ilustrado no framework clássico de [Hevner et al. \(2004\)](#) (Figura 5), a condução desta pesquisa estabelece uma sinergia entre três domínios fundamentais:

- O **Ambiente** prático da Câmara dos Deputados, que define as necessidades analíticas e fornece os dados brutos de proposições e coautorias;
- A **Base de Conhecimento** científico, que fornece as fundações teóricas (ciência de redes e teoria dos grafos) e as metodologias de desenvolvimento (princípios arquiteturais de software e testes);
- O **Espaço de Pesquisa** em si, focado na construção (implementação do *pipeline* analítico em Python) e na avaliação iterativa do artefato gerado.

Figura 5 – Framework de Pesquisa em Design Science



Fonte: Adaptado de Hevner et al. (2004, p. 80).

Para orquestrar essas frentes, a investigação estrutura-se em torno de três ciclos dinâmicos propostos na literatura de DSR (Hevner, 2007): o *ciclo de relevância*, que conecta o ambiente do problema às exigências do artefato; o *ciclo de rigor*, que fundamenta o desenvolvimento na base de conhecimento existente; e o *ciclo de design* propriamente dito, que executa iterativamente as fases de construção e avaliação até que o artefato atinja a maturidade necessária para responder ao problema analítico mapeado.

3.1.2 Fases do Método

O ciclo DSR foi instanciado neste trabalho nas seguintes fases:

- Identificação do problema:** constatação da lacuna entre a abundância de dados legislativos abertos da Câmara dos Deputados e a escassez de ferramentas analíticas reprodutíveis capazes de extrair, desses dados, evidências mensuráveis sobre estruturas de influência política;
- Definição de objetivos:** estabelecimento dos requisitos do artefato — modularidade, reprodutibilidade, cobertura por testes, interoperabilidade com ferramentas externas (Gephi), independência da fonte de dados e independência do formato de persistência, esta última assegurada por interfaces (contratos) que desacoplam o núcleo analítico dos detalhes de exportação;

- c) **Projeto e desenvolvimento:** construção iterativa do software seguindo Arquitetura Limpa (Martin, 2017) e princípios SOLID, com cobertura por testes automatizados desde o início;
- d) **Demonstração:** execução do *pipeline* completo para os anos 2022, 2023, 2024 e 2025, gerando os artefatos de dados (CSV, GEXF, SQLite e JSON);
- e) **Avaliação:** validação técnica (suíte de testes) e validação científica (teste de permutação contra modelo nulo);
- f) **Comunicação:** disponibilização do código-fonte em repositório aberto, documentação técnica e elaboração do presente trabalho.

3.1.3 Uso de Ferramentas de Inteligência Artificial

Em conformidade com as orientações de integridade acadêmica, registra-se que ferramentas de inteligência artificial generativa, baseadas em modelos de linguagem, foram utilizadas de forma auxiliar durante a elaboração deste trabalho. Seu uso englobou: apoio à revisão gramatical e estruturação textual; auxílio à compreensão e discussão de conceitos técnicos; e suporte na escrita, otimização e refatoração de trechos de código-fonte. A concepção da arquitetura de software, as decisões metodológicas e de modelagem, a validação lógica do sistema, a interpretação dos resultados e a autoria intelectual do trabalho são de responsabilidade exclusiva do autor.

3.2 PROCEDIMENTOS DE COLETA E TRATAMENTO DE DADOS

Para subsidiar a modelagem da rede parlamentar e viabilizar o cálculo de centralidades e a identificação de comunidades, estruturou-se um *pipeline* sistemático de aquisição e processamento de dados. Esse fluxo operacional compreende etapas sequenciais que vão desde o consumo automatizado das bases públicas oficiais até a persistência das redes resultantes em formatos padronizados de alta interoperabilidade. Em conformidade com as diretrizes de desacoplamento da Arquitetura Limpa, todo o fluxo de ingestão e higienização de dados é tratado de maneira isolada do núcleo analítico da aplicação. Essa separação física e lógica entre as rotinas de extração (*I/O*) e as regras de negócio assegura a integridade das métricas calculadas e garante a reprodutibilidade estrita do *pipeline* para toda a série temporal analisada, compreendida entre os anos de 2022 e 2025.

3.2.1 Fonte dos Dados

A fonte primária de dados é o Portal de Dados Abertos da Câmara dos Deputados (<https://dadosabertos.camara.leg.br>), mantido pela própria instituição em cumprimento à Lei de Acesso à Informação (Lei 12.527/2011). Trata-se da *fonte oficial primária* dos registros legislativos

brasileiros, sem autoridade superior de referência. Eventuais inconsistências da fonte representam limitação intrínseca ao registro público nacional e estão fora do escopo deste trabalho.

Foram utilizados dois conjuntos de arquivos CSV por ano analisado:

- `proposicoesAutores-YYYY.csv`: contém o vínculo entre cada proposição e seus respectivos autores, fornecendo o substrato relacional para construção das arestas;
- `proposicoes-YYYY.csv`: contém os metadados de cada proposição, incluindo o tipo (PL, PEC, PLP etc.), utilizado para aplicação dos filtros de relevância.

O download foi automatizado com cache local: arquivos baixados em uma execução são reaproveitados em execuções subsequentes, garantindo reprodutibilidade exata mesmo diante de eventuais atualizações nos arquivos do portal.

3.2.2 Recorte Temporal

A delimitação do período em 2022 a 2025 resulta de critérios técnicos, e não de conveniência. Primeiro, *estabilidade e homogeneidade do esquema de dados*: os arquivos CSV anuais do Portal de Dados Abertos mantêm formato e cobertura consistentes no período recente, evitando a heterogeneidade de esquema e a degradação de completude observadas em séries mais antigas. Segundo, *coerência institucional*: o intervalo abrange uma legislatura quase completa (a 57^a, iniciada em 2023) somada ao ano de transição (2022, encerramento da 56^a), permitindo testar a *persistência* da estrutura sob um regime legislativo estável, sem o confundimento de múltiplas mudanças de regras eleitorais e de composição partidária que um intervalo mais amplo (por exemplo, 2010 a 2025) introduziria. Terceiro, *viabilidade computacional e de auditoria*: quatro *snapshots* anuais completos são suficientes para avaliar a estabilidade do padrão estrutural (H1) mantendo cada grafo integralmente reproduzível e inspecionável. A ampliação da série temporal é registrada como trabalho futuro.

3.2.3 Ferramental de Processamento

O ecossistema técnico utilizado é integralmente baseado em software livre:

- **Python 3.11**: linguagem principal;
- **NetworkX 3.2** (Hagberg; Schult; Swart, 2008): representação, algoritmos de grafos e detecção de comunidades (Louvain e Label Propagation nativos);
- **pandas 2.2**: manipulação tabular e leitura dos CSVs;
- **pydantic** e **pydantic-settings**: configuração tipada e validada a partir do arquivo `.env`;
- **matplotlib** e **seaborn**: visualização e geração de figuras;

- **pytest** com **pytest-cov**: validação automatizada e medição de cobertura;
- **Docker** e **Docker Compose**: empacotamento reprodutível do ambiente de execução.

3.3 ESTRATÉGIA DE MODELAGEM E MITIGAÇÃO DE VIÉS

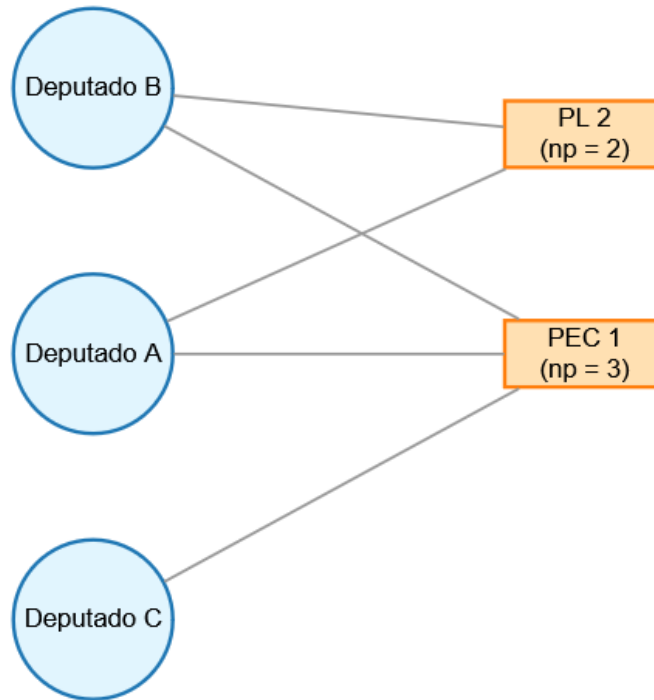
Esta seção formaliza a transformação dos dados legislativos brutos em um objeto matemático analisável e explicita as decisões tomadas para evitar que artefatos do próprio processo de modelagem sejam confundidos com estrutura política real. Primeiro define-se a topologia adotada — o grafo bipartido de autoria e sua projeção unimodal sobre o conjunto de deputados. Em seguida, trata-se do problema central dessa projeção, a inflação de arestas, por meio de uma heurística de ponderação e de filtros qualitativos sobre as proposições consideradas.

3.3.1 Definição Topológica

O ecossistema legislativo da Câmara dos Deputados forma naturalmente um *grafo bipartido* $B = (V_B, E_B)$ (Barabási, 2016; Newman, 2010), onde o conjunto de vértices é particionado em dois subconjuntos disjuntos, $V_B = D \cup P$ com $D \cap P = \emptyset$ (Diestel, 2017):

- D é o conjunto de deputados federais ativos no período analisado;
- P é o conjunto de proposições legislativas apresentadas no respectivo ano;
- E_B é o conjunto de arestas de autoria, em que uma aresta não-direcionada existe entre um deputado $d \in D$ e uma proposição $p \in P$ se e somente se o deputado d for autor ou coautor da proposição p (Diestel, 2017).

Figura 6 – Grafo bipartido de coautoria: deputados (D) e proposições (P), com n_p indicando o número de coautores de cada matéria.

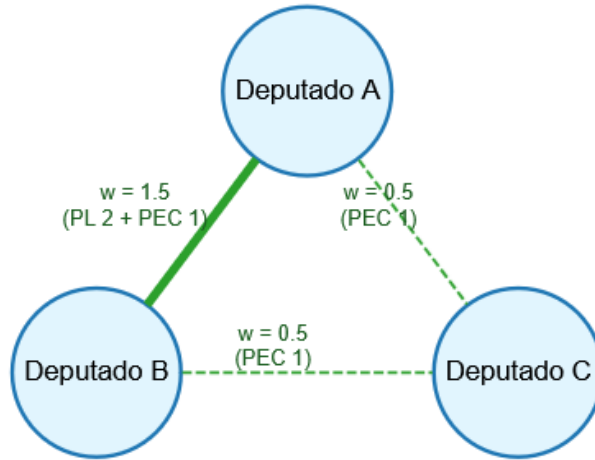


Fonte: Elaborado pelo autor usando Graphviz Online (2026).

Esse grafo bipartido é transformado, por meio de uma *projeção unimodal* (ou de modo único) sobre o conjunto de deputados D , em uma rede de coautoria unimodal $G = (D, E', w)$ (Newman, 2010; Barabási, 2016). Nessa projeção, dois deputados $i, j \in D$ são conectados por uma aresta em E' se e somente se ambos assinarem pelo menos uma proposição em comum ($p \in P$), e o peso da aresta $w(i, j)$ demonstra a intensidade dessa colaboração legislativa com base na relevância das matérias compartilhadas.

Explicita-se, para evitar ambiguidade, a natureza do objeto matemático analisado: a rede de coautoria G é um *grafo simples* (sem laços ou arestas múltiplas — coautorias repetidas entre o mesmo par acumulam-se no peso, não em arestas paralelas), *não-direcionado* (a coautoria é uma relação simétrica) e *ponderado* (cada aresta carrega o peso $w(i, j)$ da Equação 12). Cada ano é modelado como um *snapshot estático* independente; a dimensão temporal do estudo (Seção 5.4) não decorre de um grafo dinâmico único, mas da *comparação* entre os quatro grafos anuais construídos de forma idêntica.

Figura 7 – Projeção unimodal do grafo bipartido da Figura 6 sobre o conjunto D , com os pesos calculados pela Equação 12.



Fonte: Elaborado pelo autor usando Graphviz Online (2026).

A Figura 7 ilustra essa operação sobre o exemplo da Figura 6. Como os deputados A e B assinam em conjunto tanto o PL 2 ($n_p = 2$) quanto a PEC 1 ($n_p = 3$), a aresta que os conecta acumula as duas contribuições. Já os pares A–C e B–C compartilham apenas a PEC 1, recebendo uma única contribuição atenuada pelo número de signatários. O deputado C, ausente do PL 2, não estabelece com A e B a mesma intensidade de vínculo. A formalização do peso $w(i, j)$ que produz esses valores é apresentada na subseção seguinte.

3.3.2 Heurística de Ponderação e Normalização do Grafo

A definição do peso $w(i, j)$ das arestas na projeção unimodal é uma decisão metodológica crítica deste trabalho. Uma soma ingênua do número de coautorias simples mostra-se inadequada para o contexto legislativo: uma única Proposta de Emenda à Constituição (PEC), que frequentemente reúne mais de 200 signatários — bem acima do quórum de 171 assinaturas (um terço da Câmara) exigido para apresentação, geraria, sozinha, um clique completo de $\binom{200}{2} = 19.900$ arestas com peso 1, distorcendo e dominando toda a estrutura topológica subsequente.

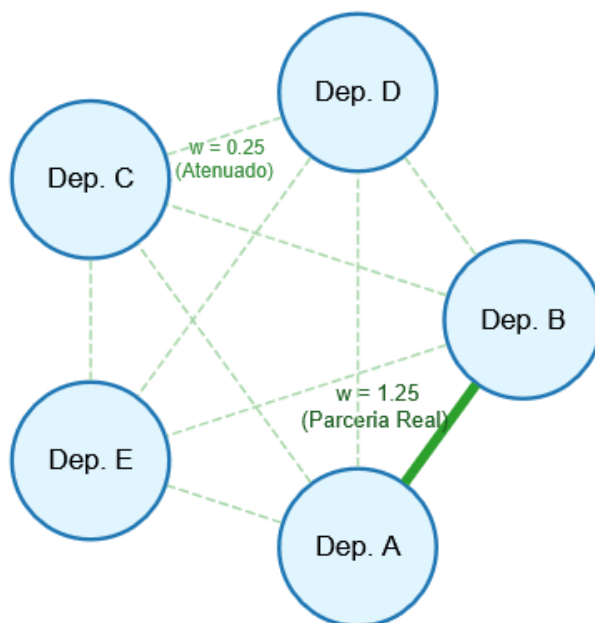
Para mitigar esse viés de escala e capturar a verdadeira intensidade da colaboração política, adota-se a metodologia de projeção ponderada proposta por [Newman \(2001\)](#), [Newman \(2010\)](#):

$$w(i, j) = \sum_{p \in P_{ij}} \frac{1}{n_p - 1} \quad (12)$$

onde P_{ij} representa o conjunto de proposições legislativas assinadas em comum por ambos os deputados i e j , e n_p é o número total de autores da proposição p , com a restrição de que $n_p \geq 2$ para todas as matérias em P_{ij} (uma vez que coautorias exigem múltiplos autores). Sob essa formulação, a contribuição individual de cada proposição para a força da conexão decresce de forma inversamente proporcional ao seu número de signatários. Isso permite equilibrar a influência de coautorias em pequenos grupos (que possuem alta especificidade e indicam forte

articulação política) e em grandes blocos de assinaturas (que possuem baixa especificidade analítica).

Figura 8 – Efeito da heurística de ponderação na projeção unimodal.



Fonte: Elaborado pelo autor usando Graphviz Online (2026).

A Figura 8 ilustra o efeito prático da Equação 12. Considere uma PEC assinada pelos cinco deputados do exemplo ($n_p = 5$) e um Projeto de Lei assinado exclusivamente por A e B ($n_p = 2$). A PEC de massa conecta todos os $\binom{5}{2} = 10$ pares, mas com peso atenuado de $1/(5 - 1) = 0,25$ cada; o PL específico acrescenta $1/(2 - 1) = 1,00$ apenas ao par A–B. O resultado é que a articulação política efetiva entre A e B ($w = 1,25$) destaca-se em cinco vezes sobre as conexões geradas pela assinatura coletiva ($w = 0,25$) — precisamente o comportamento que a heurística busca produzir.

Adicionalmente, três decisões qualitativas complementam a heurística:

- a) **Filtro de tipos de proposição:** Apenas proposições com conteúdo legislativo substantivo são consideradas: Projetos de Lei (PL), Propostas de Emenda à Constituição (PEC), Projetos de Lei Complementar (PLP), Projetos de Decreto Legislativo (PDL) e Emendas (EMC). Requerimentos, ofícios e proposições administrativas são excluídos.
- b) **Filtro de massa de signatários:** Proposições com mais de 30 deputados coautores são excluídas da construção de arestas. Empiricamente, a inclusão dessas matérias — a maior delas uma PEC com 226 signatários — eleva a densidade da rede de 9,9% para 85,4% em 2025, transformando-a em um grafo quase-completo e inviabilizando a detecção de comunidades (demonstração no Capítulo 5). Como consequência direta, **nenhuma PEC integra a rede de coautoria analisada**, pois todas superam o limiar de 30 — resultado intencional, e não uma perda: a co-assinatura em massa reflete adesão agregada, não

articulação coordenada, e sua exclusão é coerente com a definição de influência adotada. Cabe notar que a heurística de ponderação $1/(n_p - 1)$ e este filtro são **complementares**: a primeira trata a distorção de *peso* (cada par de uma PEC de massa contribui apenas $1/225$), enquanto o filtro trata a distorção *topológica* — a mera *existência* das $\binom{226}{2} \approx 25$ mil arestas, que as métricas topológicas (intermediação, proximidade) e a própria densidade não conseguem ignorar. O limite de 30 é uma decisão conservadora, declarada como parâmetro sensível para análise de sensibilidade em trabalhos futuros.

- c) **Pesos uniformes por tipo de proposição**: Embora que durante o desenvolvimento desse trabalho tenha-se experimentado pesos diferenciados por tipo (por exemplo, PEC com peso maior que PL), não existe na literatura uma escala teoricamente justificada para tal ponderação. Adota-se, portanto, peso igual a 1 para todos os tipos, sendo a seleção qualitativa realizada pelo filtro de tipos descrito acima.

3.4 SELEÇÃO DE ALGORITMOS E MÉTRICAS ANALÍTICAS

A seleção das ferramentas analíticas orienta-se por dois critérios: consolidação na literatura e disponibilidade de implementações maduras e auditáveis, requisito derivado do compromisso de reprodutibilidade deste trabalho. As subseções a seguir tratam, respectivamente, das métricas de centralidade adotadas e dos algoritmos de detecção de comunidades.

3.4.1 Métricas de Centralidade

Quatro métricas de centralidade são computadas para cada deputado em cada ano, conforme definições formais apresentadas na Seção 2.2.4.

Antes de aplicar essas métricas, é necessário entender o que se entende por influência neste trabalho. *Influência política é aqui operacionalizada, em sentido estritamente estrutural, como a posição de um deputado na rede de coautoria* — e não como poder institucional formal (relatorias, presidência de comissões, controle de pauta), que se exerce por mecanismos não capturados pela coautoria. Especificamente, a influência estrutural é mensurada por duas dimensões complementares: a intermediação (*betweenness*), que capta o papel de ponte entre grupos, na tradição de Freeman (1977); e a centralidade de autovetor, que capta a inserção em vizinhanças bem conectadas (Bonacich, 1972). Trata-se, portanto, de uma medida de articulação e colaboração legislativa, não de poder decisório — distinção retomada na discussão dos resultados.

Dessa forma, cada métrica captura uma dimensão distinta de influência: grau pondera quantidade de colaborações; intermediação captura papel de ponte entre blocos; proximidade mede capacidade de disseminação; autovetor pondera a qualidade das conexões. Estas métricas compõem a caracterização descritiva das estruturas de influência da rede, apresentada no Capítulo 5.

Cada métrica emprega, ademais, a representação coerente com sua semântica. Grau e autovetor são calculados sobre o grafo *ponderado*, pois a intensidade da colaboração (afinidade) acumula-se de forma aditiva. Já intermediação e proximidade são calculadas sobre a topologia *não-ponderada*: em grafos ponderados, o peso é interpretado como distância ou custo, ao passo que na rede de coautoria o peso representa afinidade — semântica inversa, cuja conciliação exigiria uma transformação arbitrária $1/w$. A intermediação é *normalizada* pelo fator $(n - 1)(n - 2)/2$ (grafos não-direcionados), produzindo valores em $[0, 1]$ comparáveis entre anos de tamanhos distintos, requisito da análise temporal (Seção 5.4).

3.4.2 Detecção de Comunidades

Dois algoritmos são aplicados em paralelo:

- a) **Louvain** (Blondel et al., 2008): otimização gulosa da modularidade, executado com semente fixa ($seed = 42$) para garantir reprodutibilidade;
- b) **Label Propagation** (Raghavan; Albert; Kumara, 2007): propagação local de rótulos, utilizado como contraprova.

O Louvain é considerado o método primário; o Label Propagation funciona como verificação de robustez — quando ambos identificam estruturas similares, há maior confiança no achado.

3.5 MEDIDA DE ROBUSTEZ: ADJUSTED RAND INDEX (ARI)

Para avaliar a robustez da detecção de comunidades — e, por consequência, a solidez da hipótese central, compara-se a partição obtida pelo Louvain com a obtida pelo Label Propagation por meio do *Adjusted Rand Index* (ARI) (Hubert; Arabie, 1985). O ARI mede a similaridade entre duas partições de um mesmo conjunto de nós, corrigida para a concordância esperada ao acaso: o valor 1 indica partições idênticas (a menos de relabelagem), 0 indica concordância ao nível do acaso e valores negativos indicam concordância inferior à aleatória. O índice é computado a partir da tabela de contingência entre as duas partições, e um ARI elevado é interpretado como evidência de que a estrutura comunitária detectada é robusta à escolha do algoritmo.

3.6 AVALIAÇÃO DO ARTEFATO (PROCEDIMENTOS DE VALIDAÇÃO)

A avaliação do artefato ocorre em duas frentes complementares, alinhadas à fase de avaliação do ciclo DSR. A validação qualitativa confronta os achados com a literatura sobre comportamento legislativo brasileiro; a quantitativa combina teste estatístico de significância, cobertura por testes automatizados e reprodutibilidade do ambiente de execução.

3.6.1 Validação Qualitativa

A validação qualitativa é realizada por meio da inspeção dos resultados frente à literatura política e ao conhecimento institucional sobre a Câmara dos Deputados. Especificamente, a hipótese detalhada no Capítulo 1 é confrontada com os achados empíricos.

3.6.2 Validação Quantitativa e Reprodutibilidade

A validação quantitativa é executada em três níveis:

- a) **Teste de permutação contra modelo nulo:** Para verificar se a modularidade Q_{obs} observada é estatisticamente significativa, são gerados 200 grafos aleatórios por meio do procedimento *double-edge-swap*, que preserva exatamente a sequência de graus do grafo original mas randomiza a topologia. Como o *double-edge-swap* redistribui as arestas sem seus pesos, o conjunto (*multiset*) de pesos originais é reatribuído às arestas reconfiguradas, preservando a distribuição de pesos da rede real e assegurando que a comparação seja ponderada e homogênea com Q_{obs} . O p -value é estimado pelo estimador aditivo $p = (r + 1)/(m + 1)$, recomendado por [North, Curtis & Sham \(2002\)](#), em que r é o número de grafos nulos com $Q_{\text{nulo}} \geq Q_{\text{obs}}$ e m é o número de permutações — evitando que seja exatamente nulo, o que seria estatisticamente inválido em uma amostra finita. Adota-se nível de significância $\alpha = 0,05$.
- b) **Suíte de testes automatizados:** O artefato de software é validado por 217 testes automatizados (cobertura total 81%, núcleo analítico $\geq 93\%$), incluindo: (i) testes unitários sobre entidades de domínio e algoritmos; (ii) testes de integração sobre fluxos que combinam múltiplos componentes — a cadeia de processamento dos dados brutos, a construção e análise do grafo e a persistência com ida-e-volta (serialização e desserialização) —; (iii) testes de integridade sobre os *datasets* gerados, parametrizados por ano (verificação de esquema, faixas de valores plausíveis, coerência CSV $\leftrightarrow$ GEXF).
- c) **Reprodutibilidade containerizada:** O ambiente de execução completo é encapsulado em imagem Docker, permitindo replicação exata em qualquer máquina com Docker instalado, independente de sistema operacional ou versões locais de bibliotecas.

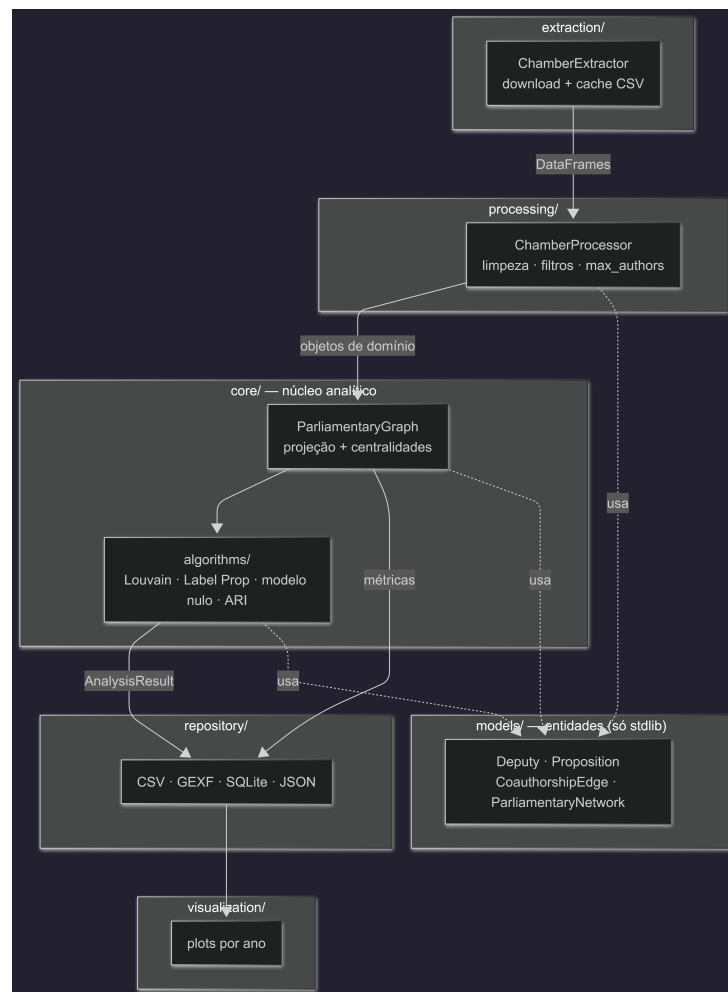
4 DESENVOLVIMENTO

Este capítulo descreve a instânciação concreta do artefato de software — denominado *pipeline* — conforme os princípios arquiteturais e a metodologia descritos nos capítulos anteriores.

4.1 DEFINIÇÃO DA ARQUITETURA DO SISTEMA E ESTRUTURA MODULAR

O *pipeline* é organizado em camadas com responsabilidades únicas, conforme o esquema:

Figura 9 – Diagrama de componentes e fluxo de dados do *pipeline*.



Fonte: Elaborado pelo autor.

A dependência entre camadas respeita a Regra da Dependência da Arquitetura Limpa: camadas inferiores (extraction, repository) dependem das superiores (models, core), nunca o contrário. Cada camada é um pacote Python independente, contido em `src/`.

4.1.1 Models (Entidades)

A camada `models/` contém as entidades de domínio puro, implementadas como *dataclasses* do Python (estruturas imutáveis-por-convenção com sintaxe declarativa). Não há dependência de bibliotecas externas além da biblioteca padrão.

- `Deputy`: representa um deputado federal, com identificador único, nome, sigla do partido, sigla da unidade federativa, os cinco campos de métricas de centralidade computadas e a comunidade Louvain atribuída (`community_louvain`);
- `Proposition`: representa uma proposição legislativa, com identificador, ano, lista de identificadores de autores e tipo (PL, PEC etc.);
- `CoauthorshipEdge`: representa uma aresta da rede de coautoria, com identificadores de origem e destino, peso bruto e peso normalizado;
- `ParliamentaryNetwork`: agregador que contém o grafo `NetworkX` construído e o dicionário de deputados, fornecendo a interface canônica para as camadas analíticas.

4.1.2 Core (Casos de Uso)

A camada `core/` é o núcleo analítico do sistema, contendo a classe `ParliamentaryGraph` e os submódulos `algorithms/`. É a camada de mais alta criticidade — justificando a cobertura por testes $\geq 93\%$.

A classe `ParliamentaryGraph` encapsula:

- Construção do grafo a partir das estruturas de coautoria, aplicando a fórmula de ponderação apresentada na Equação 12;
- Cálculo das quatro métricas de centralidade descritas na metodologia;
- Atribuição das comunidades detectadas aos objetos `Deputy` (`assign_communities`);
- Resolução de *aliases* de identificadores (deputados com múltiplos IDs no sistema da Câmara).

O submódulo `algorithms/` agrupa funções puras, isoladas de qualquer preocupação de entrada e saída:

- `metrics.py`: cálculo de centralidades e normalização;
- `community_detection.py`: implementações encapsuladas dos algoritmos Louvain e Label Propagation, com cálculo de modularidade;
- `validation.py`: teste de permutação contra modelo nulo, que operacionaliza a hipótese central (H1);

- `analysis_result.py`: *dataclass* agregadora que compõe todos os resultados analíticos de um ano em um objeto tipado, incluindo o cálculo do *Adjusted Rand Index*.

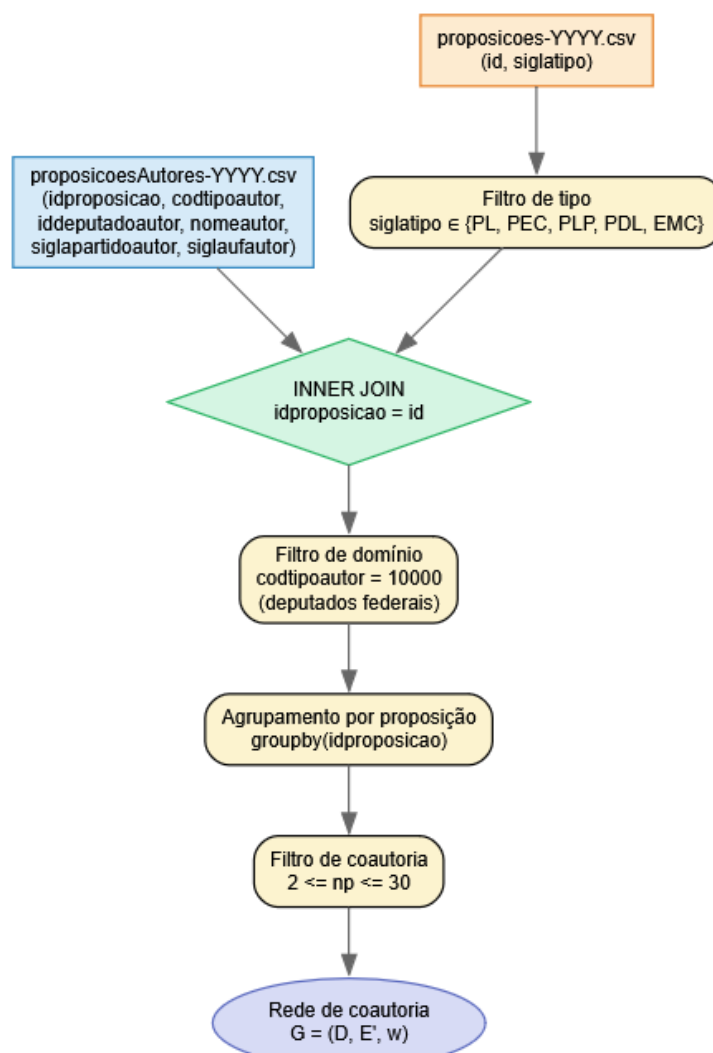
4.1.3 Extraction e Processing (Adaptadores de Interface)

A camada `extraction/` contém a classe `ChamberExtractor`, responsável pelo download e cache dos CSVs do Portal de Dados Abertos. Implementa: (i) verificação de existência local antes do download, reutilizando o cache quando presente; (ii) download direto dos arquivos CSV do portal via `pandas`; (iii) persistência do arquivo no diretório de cache local; (iv) reaproveitamento determinístico do cache em execuções subsequentes, tornando a série reprodutível sem depender da rede.

A camada `processing/` contém a classe `ChamberProcessor`, responsável por:

1. Padronização das colunas (minúsculas, sem espaços);
2. Filtro de tipos de proposição (PL, PEC, PLP, PDL, EMC);
3. *Inner join* entre autores e proposições filtradas;
4. Filtro de domínio (apenas deputados federais, código de tipo de autor 10000);
5. Construção da estrutura de coautorias, agrupada por proposição;
6. Aplicação do filtro de massa de signatários (`max_authors=30`);
7. Sanitização de valores nulos em sigla de partido e UF (substituídos por sentinelas "S/PARTIDO" e "S/UF");
8. Conversão das estruturas tabulares em objetos de domínio.

Figura 10 – Fluxo de junção e filtragem dos arquivos CSV até a construção da rede de coautoria.



Fonte: Elaborado pelo autor usando Graphviz Online (2026).

A Figura 10 ilustra as fases das operações. O cruzamento entre os dois arquivos é realizado por um *inner join* sobre a chave comum — o identificador da proposição, nomeado `idproposicao` no arquivo de autores e `id` no de metadados. Essa operação atua como filtro atômico: ao restringir previamente os metadados aos cinco tipos de proposição de interesse, o join descarta automaticamente dezenas de milhares de requerimentos, ofícios e matérias administrativas, sem necessidade de filtragem posterior. As etapas subsequentes de filtragem por número de signatários são justificadas na Subseção 3.3.2.

4.1.4 Repository (Frameworks e Drivers)

A camada `repository/` contém quatro adaptadores de persistência:

- `CsvRepository`: exporta métricas de deputados e arestas para arquivos CSV, ordenados por centralidade decrescente, incluindo a coluna de comunidade;

- `GraphExporter`: exporta o grafo `NetworkX` no formato GEXF, padrão de interoperabilidade com a ferramenta Gephi;
- `DB_Exporter`: persiste métricas em banco SQLite, com operação *upsert* (insere ou atualiza por chave composta ano + ID) e migração idempotente de esquema via `ALTER TABLE`;
- `AnalysisRepository`: serializa o resultado analítico agregado do ano (`AnalysisResult`) em arquivo JSON legível e auditável, e o recarrega de forma tipada quando necessário.

Para concretizar o Princípio da Inversão de Dependência (Seção 2.1.2), esses adaptadores não são referenciados diretamente pelas camadas superiores: cada um implementa um *contrato* (interface) declarado no módulo `repository/interfaces.py` por meio de *protocolos* (`typing.Protocol`). A Listagem 4.1 apresenta os contratos definidos.

Listing 4.1 – Contratos da camada de persistência (`repository/interfaces.py`).

```

1 class MetricsRepository(Protocol):
2     def export_deputy_metrics(self, deputies, year) -> Path: ...
3     def export_coauthorship_metrics(self, coauthorships, year) ->
      Path: ...
4
5 class DatabaseRepository(Protocol):
6     def export_deputy_metrics(self, deputies, year) -> Path: ...
7
8 class GraphRepository(Protocol):
9     def export_gexf(self, graph, year=None) -> Path: ...
10
11 class AnalysisStore(Protocol):
12     def save(self, result) -> Path: ...
13     def load(self, year): ...

```

A orquestração do *pipeline* depende dessas abstrações, e não das classes concretas: a estrutura `PipelineDependencies` declara seus colaboradores de persistência pelos tipos dos protocolos, e a instanciação das implementações concretas ocorre em um único ponto de composição (`main.py`). Como consequência, trocar o formato de exportação (por exemplo, substituir o CSV por Parquet, ou o SQLite por outro banco) exige apenas fornecer uma nova classe que satisfaça o mesmo contrato, sem qualquer alteração no núcleo analítico. Trata-se de *injeção de dependências* no ponto de composição, técnica que operacionaliza a inversão de controle e mantém o núcleo (`core/`) independente de detalhes de I/O — coerente com a Regra da Dependência, verificável no código: o pacote `core/` não importa nenhuma das camadas externas.

Adicionalmente, a camada `visualization/` consome os dados gerados e produz figuras (PNG) e relatório textual (TXT) por ano, organizados em subdiretórios `data/plots/{ano}/`.

4.2 MODELAGEM DO GRAFO PARLAMENTAR E HEURÍSTICA DE PONDERAÇÃO

A construção do grafo segue a fórmula apresentada na Equação 12, implementada na classe `ParliamentaryGraph`. Para cada proposição $p \in P_{\text{filtrado}}$ com n_p coautores:

1. Gera-se o conjunto $\{(i, j) : i, j \in \text{autores}(p), i < j\}$ de pares ordenados;
2. Para cada par, adiciona-se $1/(n_p - 1)$ ao peso acumulado da aresta no grafo.

4.2.1 Tipos de Proposição Considerados

Os cinco tipos admitidos pelo filtro qualitativo (Seção 3.3.2) têm perfis distintos de coautoria, o que justifica tratá-los individualmente:

- a) **Projetos de Lei (PL)**: a categoria mais numerosa e diversa do *dataset*, com ampla variabilidade no número de coautores — de proposições individuais a iniciativas com dezenas de signatários. Sob a heurística adotada, PLs com poucos coautores contribuem mais fortemente para as arestas, refletindo coordenação política mais específica;
- b) **Projetos de Lei Complementar (PLP)**: por exigirem maioria absoluta para aprovação e tratarem de matérias constitucionalmente reservadas, tendem a apresentar coautorias mais selecionadas; numericamente menos frequentes que os PLs, mas com contribuição estrutural comparativamente densa;
- c) **Propostas de Emenda à Constituição (PEC)**: os instrumentos de maior peso simbólico e os principais responsáveis pelo viés de massa de signatários, por exigirem 171 assinaturas (um terço da Câmara) para apresentação; a heurística de ponderação e o filtro de massa (Seção 3.3.2) mitigam e eliminam, respectivamente, esse efeito distorcivo sobre a topologia da rede;
- d) **Projetos de Decreto Legislativo (PDL)**: tratam de matérias de competência exclusiva do Congresso Nacional que dispensam sanção presidencial — por exemplo, a aprovação de tratados internacionais e a sustação de atos normativos do Poder Executivo. Apresentam volume reduzido frente aos PLs, com padrão de coautoria mais institucional e menos numeroso;
- e) **Emendas (EMC)**: modificações propostas a uma proposição já em tramitação, tipicamente na fase de comissão. Por incidirem sobre um texto específico e não constituírem matéria autônoma, concentram-se predominantemente em poucos autores.

4.3 IMPLEMENTAÇÃO DO NÚCLEO ANALÍTICO COMPUTACIONAL

Definida a modelagem do grafo, esta seção detalha a instanciação do núcleo analítico na camada `core/`, onde reside a lógica de mais alta criticidade do artefato. Coerentemente com a Regra da Dependência, esse núcleo é isolado de qualquer preocupação de entrada e saída: recebe o grafo já construído e devolve resultados tipados, sem conhecer a origem dos dados nem o destino das saídas. As subseções a seguir descrevem, nessa ordem, o cálculo das quatro métricas de centralidade, a detecção de comunidades pelos dois algoritmos adotados e a persistência dos resultados analíticos em um objeto único e auditável.

4.3.1 Cálculo de Centralidades

As centralidades são computadas via funções da biblioteca NetworkX, que implementa internamente algoritmos clássicos:

- Grau ponderado: `graph.degree(weight="weight")`;
- Intermediação: algoritmo de [Brandes \(2001\)](#), complexidade $O(|V| \cdot |E|)$ em grafos esparsos;
- Proximidade: implementação sobre a topologia não-ponderada, coerente com a justificativa da Seção 2.2.4;
- Autovetor: método de potência iterativo até convergência.

Os valores normalizados são atribuídos aos campos correspondentes do objeto `Deputy` via método `compute_all_centralities`, mantendo a integridade referencial entre o grafo e a coleção de entidades.

4.3.2 Detecção de Comunidades

A detecção de comunidades é encapsulada na classe `CommunityDetector`, que expõe três métodos, conforme ilustrado na Listagem 4.2:

Listing 4.2 – Interface da classe `CommunityDetector`.

```

1  class CommunityDetector:
2      def detect_louvain(self, graph, seed=42) -> dict:
3          """Returns {node_id: community_id}"""
4
5      def detect_label_propagation(self, graph) -> dict:
6          """Returns {node_id: community_id}"""
7
8      def calculate_modularity(self, graph, partition) -> float:
9          """Returns Q value for a given partition"""

```

A semente `seed=42` fixa a sequência pseudo-aleatória interna do Louvain, garantindo que execuções repetidas com o mesmo grafo produzam exatamente a mesma partição — requisito fundamental para reprodutibilidade científica.

A validação por modelo nulo é implementada em `algorithms/validation.py`, na função `assess_community_significance`, que executa 200 iterações de *double-edge-swap* (preservando a sequência de graus), recalcula Louvain em cada iteração, e retorna um objeto `NullModelResult` com as estatísticas agregadas.

4.3.3 Persistência dos Resultados Analíticos

Ao final da etapa analítica de cada ano, todos os resultados — métricas topológicas, resumos das partições Louvain e Label Propagation, o *Adjusted Rand Index* e o resultado do modelo nulo (H1) — são compostos em um objeto tipado `AnalysisResult` e serializados pela `AnalysisRepository` em `data/analysis/analysis_YYYY.json`. Esse arquivo, além de conter os metadados de execução (*timestamp*, *max_authors*, *n_permutations*), é a fonte única de verdade consumida tanto pela análise comparativa entre anos quanto pela elaboração das tabelas do Capítulo 5, eliminando a necessidade de reexecutar o *pipeline* para obter qualquer número reportado.

4.4 QUALIDADE, INFRAESTRUTURA E REPRODUTIBILIDADE

A confiabilidade do artefato — requisito central no paradigma DSR — é sustentada por três pilares descritos nesta seção: uma suíte de testes automatizados que valida tanto o código quanto os dados por ele gerados; o empacotamento do ambiente de execução em contêiner Docker, que assegura replicação exata; e a disponibilização do código-fonte em repositório aberto, condição para a auditabilidade e o reuso do trabalho por terceiros.

4.4.1 Suíte de testes

O sistema é coberto por 217 testes automatizados, distribuídos em módulos que cobrem entidades de domínio, construção e análise do grafo, algoritmos de comunidade, validação estatística por modelo nulo, repositórios de persistência e integridade dos *datasets* gerados. Um subconjunto de 64 testes de integridade é parametrizado por ano e verifica, sobre os arquivos efetivamente produzidos pelo *pipeline*, a conformidade de esquema, faixas de valores plausíveis, ausência de nulos críticos, coerência CSV ↔ GEXF e plausibilidade estatística (densidade, modularidade, componente principal).

A cobertura total é de 81%, com o núcleo analítico (`core/`) atingindo $\geq 93\%$ em todos os submódulos. Os testes de integridade de *datasets* representam contribuição metodológica relevante: além de validar o código, validam empiricamente as características dos dados produzidos, capturando regressões silenciosas que testes unitários não detectariam.

Cabe uma ressalva de transparência sobre esses números. Dos 217 testes, os 64 de integridade dependem dos artefatos de dados: só executam após o *pipeline* gerar o diretório `data/`. Em um *checkout* limpo, sem essa geração prévia, executam os 153 testes restantes; a cobertura de 81% pressupõe, portanto, os artefatos de dados já produzidos. Trata-se de uma decisão consciente — validar também os dados, e não apenas o código —, cujo custo é essa dependência de ambiente, mitigada pela reprodutibilidade containerizada.

4.4.2 Empacotamento Docker

O ambiente de execução completo é definido em três arquivos:

- `Dockerfile`: imagem base Python 3.11-slim, instalação automática de dependências;
- `docker-compose.yml`: três serviços — `pipeline_chamber` (executa o *pipeline* completo), `compare` (executa análise comparativa entre anos), `tests` (executa a suíte de testes);
- `requirements.txt`: lista de bibliotecas Python com versões fixadas.

A execução do *pipeline* completo para os quatro anos (2022–2025) é desencadeada por um único comando: `docker compose up pipeline_chamber`. O tempo médio de execução em hardware de classe *desktop* (Intel i5-14600KF, 32 GB DDR5) é de 15 a 25 minutos.

4.4.3 Orquestração do projeto

O ambiente pode ser orquestrado totalmente pelo shell `run.sh`, assim como a execução pode ser feita por esse mesmo shell. Ademais, o comando `./run.sh help` pode ser de utilidade para saber os comandos específicos do shell.

4.4.4 Repositório aberto

O código-fonte completo do artefato, juntamente com a documentação técnica e os arquivos de empacotamento, está disponível publicamente em repositório aberto (<https://github.com/fvilhalva/parliament-graph-architecture>), em conformidade com os princípios de *Open Science*. A disponibilização do código não é um adendo, mas parte constitutiva da contribuição: é o que torna os resultados auditáveis, verificáveis e extensíveis por terceiros, fechando o ciclo de reprodutibilidade proposto pelo trabalho.

4.4.5 Integração Contínua e Controle de Versão

A garantia de qualidade não se restringe à existência da suíte de testes: ela é aplicada automaticamente a cada alteração do código, por meio de *Integração Contínua* (CI). O repositório define um fluxo de trabalho do *GitHub Actions* (`.github/workflows/ci.yml`) que, a cada

push ou *pull request* destinado aos ramos `main` e `develop`, provisiona um ambiente limpo com Python 3.11, instala as dependências com versões fixadas (`requirements.txt`), prepara o arquivo de configuração e executa a suíte de testes com medição de cobertura, publicando o relatório (`coverage.xml`) como artefato. Cabe uma nota de transparência coerente com a Seção 4.4.1: por executar em um *checkout* limpo, sem os artefatos de dados, a CI roda o subconjunto de testes independentes de `data/` (os 153 que não dependem do *pipeline*); a suíte completa, com os testes de integridade de *dataset*, é exercida quando os dados já foram gerados.

O desenvolvimento seguiu, ainda, um fluxo de ramificação disciplinado — `main` → `develop` → `feat/*` —, com a integração de cada funcionalidade mediada por *pull requests*. Esse fluxo é reforçado por regras de proteção configuradas no repositório (*rulesets*): o ramo `main` não aceita envio direto — toda integração exige um *pull request* cuja verificação de CI (o *status check* da suíte de testes) tenha sido bem-sucedida —, além de bloquear reescrita de histórico (*force-push*) e exclusão do ramo. Em conjunto, CI e proteção de ramo asseguram, por construção, que não seja possível integrar ao ramo principal código que não passe na verificação automatizada — uma evidência concreta do compromisso com a reprodutibilidade e a integridade do artefato.

5 RESULTADOS E DISCUSSÕES

Este capítulo apresenta os resultados produzidos pelo artefato para o período 2022–2025 e discute seu significado. Primeiro valida-se a arquitetura e sua reprodutibilidade; em seguida, caracterizam-se a rede e os deputados mais centrais; então testa-se a hipótese central (H1) por meio da detecção de comunidades e do modelo nulo; por fim, avalia-se a estabilidade do padrão ao longo dos quatro anos.

5.1 VALIDAÇÃO DA ARQUITETURA E REPRODUTIBILIDADE

Esta seção detalha a validação da arquitetura e a reprodutibilidade do artefato de software.

5.1.1 Desempenho Computacional

O *pipeline* completo (quatro anos) executa em 15 a 25 minutos em hardware de classe *desktop* (Intel i5-14600KF, 32 GB DDR5), de forma determinística: a fixação da semente aleatória (`seed=42`) garante que execuções repetidas produzam resultados idênticos. A suíte de 217 testes automatizados (cobertura total de 81%, núcleo $\geq 93\%$) valida a corretude de cada módulo. O empacotamento em imagem Docker replica o ambiente de execução de forma exata — mesma versão de Python, das bibliotecas e do sistema base —, de modo que qualquer máquina com Docker reproduz integralmente os números reportados neste capítulo, satisfazendo o requisito de reprodutibilidade estrita do artefato.

5.1.2 Geração de Artefatos (Interoperabilidade)

Cada execução persiste os resultados em formatos abertos e interoperáveis: CSV (métricas por deputado), GEXF (grafo, compatível com o Gephi), SQLite (persistência consultável) e JSON (`AnalysisResult` auditável). O formato GEXF cumpre um papel adicional de *validação cruzada independente*: ao importar o grafo no Gephi e recalcular a modularidade com a implementação própria da ferramenta, obtêm-se valores de Q compatíveis com os produzidos pelo *pipeline*, o que confirma que os resultados não são artefato de uma única implementação. A pluralidade de formatos assegura, ainda, que os dados possam ser auditados e reanalisados por terceiros sem depender do código-fonte original.

5.1.3 Avaliação da Arquitetura: Desacoplamento e Extensibilidade

Coerentemente com o paradigma DSR — em que o artefato é a própria contribuição —, avaliam-se aqui as qualidades estruturais do projeto, e não apenas seu desempenho. Três propriedades da arquitetura, afirmadas no Capítulo 4, confirmam-se de forma objetiva.

Primeiro, o *desacoplamento* é verificável no próprio código: o núcleo analítico (`core/`) não importa nenhuma das camadas externas (extração, processamento, repositório ou visualização), respeitando estritamente a Regra da Dependência. Esse isolamento não é um detalhe estético — é o que torna o núcleo *testável de forma independente* de banco de dados, arquivos ou rede, e explica por que a cobertura de testes do `core/` atinge $\geq 93\%$: a alta testabilidade é consequência direta do design, e não um esforço acidental.

Segundo, a *extensibilidade* decorre da inversão de dependência (Seção 2.1.2), materializada pelos contratos da camada de persistência (Listagem 4.1). Como o *pipeline* depende das abstrações, e não das implementações concretas, substituir um formato de saída — por exemplo, trocar o CSV por Parquet, ou o SQLite por outro banco — exige apenas fornecer uma nova classe que satisfaça o mesmo contrato, sem qualquer alteração no núcleo analítico. O artefato é, portanto, extensível por construção, e não apenas em intenção.

Terceiro, a *corretude é reimposta automaticamente*: a integração contínua (Seção 4.4.5) executa a suíte de testes a cada alteração, e a proteção do ramo principal impede a integração de código que não passe nessa verificação. A garantia de qualidade deixa de depender de disciplina manual e passa a ser assegurada pelo próprio fluxo de desenvolvimento.

Em conjunto, essas três evidências demonstram que a arquitetura não é apenas descrita, mas *avaliada*: o artefato realiza os requisitos de modularidade, testabilidade e extensibilidade que motivaram seu projeto — o que constitui, no paradigma DSR, a demonstração de sua eficácia.

5.2 ANÁLISE DE CENTRALIDADE E INFLUÊNCIA PARLAMENTAR

Esta seção caracteriza a rede de coautoria produzida pelo artefato em dois planos. Primeiro, descreve-se a estrutura global do grafo em cada ano e o efeito empírico dos filtros de modelagem; em seguida, identificam-se os deputados que ocupam as posições estruturalmente mais centrais e discute-se o que essas posições revelam — e o que *não* revelam — sobre a articulação legislativa.

5.2.1 O Grafo Resultante

A Tabela 1 sintetiza a estrutura da rede em cada ano.

Tabela 1 – Métricas estruturais da rede de coautoria por ano.

Indicador	2022	2023	2024	2025
Nós (deputados ativos)	273	360	337	331
Arestas (coautorias únicas)	3.085	5.082	4.426	5.407
Densidade	8,31%	7,86%	7,82%	9,90%
Modularidade Louvain (Q)	0,596	0,640	0,634	0,634
Comunidades (Louvain)	15	12	14	16

Fonte: Elaborado pelo autor.

A relevância dos filtros de modelagem (Seção 3.3.2) é confirmada empiricamente. A Tabela 2 mostra o efeito do filtro de massa de signatários sobre a densidade em 2025.

Tabela 2 – Efeito do filtro de massa de signatários sobre a densidade (2025).

Configuração	Arestas	Densidade
Sem filtro de massa (apenas tipo)	124.671	85,35%
Com filtro (<code>max_authors=30</code>)	5.407	9,90%

Fonte: Elaborado pelo autor.

A remoção de 70 proposições com mais de 30 coautores — a maior com 226 signatários (uma PEC) — reduz a densidade de 85,35% para 9,90%, transformando um grafo quase-completo em uma rede analisável. Vale reforçar que a heurística de ponderação $1/(n_p - 1)$ e o filtro de massa atacam distorções *distintas e complementares*: a normalização corrige a distorção de *peso* (cada par de uma PEC de massa contribui apenas $1/225 \approx 0,004$ à força do vínculo), enquanto o filtro corrige a distorção *topológica* — a mera *existência* das dezenas de milhares de arestas de um clique de signatários, que as métricas topológicas (intermediação, proximidade) e a própria densidade não conseguem ignorar, por mais baixo que seja o peso.

A robustez do limiar de 30 e a necessidade do filtro foram avaliadas por uma análise de sensibilidade (Tabela 3), variando `max_authors` em 2025.

Tabela 3 – Sensibilidade da rede ao limiar `max_authors` (2025).

<code>max_authors</code>	Arestas	Densidade	Q Louvain
20	2.415	5,8%	0,699
30 (adotado)	5.407	9,9%	0,634
40	7.521	12,0%	0,601
Sem filtro	124.671	85,4%	0,215

Fonte: Elaborado pelo autor.

Dois resultados sobressaem. Primeiro, a modularidade é *estável* na faixa de limiares plausíveis (20 a 40), mantendo-se sempre bem acima do patamar de estrutura significativa ($Q > 0,3$); a escolha de 30 é, portanto, um meio-termo conservador que *não determina* o resultado. Segundo, e mais importante, sem o filtro — e mesmo com a normalização ativa — a modularidade *colapsa* de 0,634 para 0,215: a prova empírica de que o filtro *não é redundante* com a ponderação, pois trata uma distorção (a topológica) que a ponderação não alcança. Os valores das Tabelas 2 e 3 são gerados de forma reprodutível pelo utilitário `scripts/sensitivity.py` (comando `./run.sh sensitivity`), que reconstrói a rede sob cada configuração de `max_authors`.

5.2.2 Os Deputados mais Influentes

A Tabela 4 apresenta os deputados de maior centralidade em 2025, sob duas óticas: intermediação (pontes entre grupos) e grau ponderado (intensidade de colaboração).¹

Tabela 4 – Deputados de maior centralidade em 2025 (identidades anonimizadas).

Top intermediação (pontes)	Top grau ponderado (coesão)
Deputado 1 (Partido A)	Deputado 6 (Partido E)
Deputado 2 (Partido B)	Deputado 7 (Partido F)
Deputado 3 (Partido B)	Deputado 8 (Partido E)
Deputado 4 (Partido C)	Deputado 9 (Partido F)
Deputado 5 (Partido D)	Deputado 10 (Partido B)

Fonte: Elaborado pelo autor.

O primeiro padrão diz respeito às *pontes*. Os deputados de maior intermediação distribuem-se por *quatro partidos distintos*, e o principal *broker* não pertence ao maior bloco. O papel de ponte entre grupos é, portanto, uma posição *estrutural e individual*, não determinada pelo tamanho da bancada: um parlamentar de partido médio pode ocupar posição de mediação mais central que membros de blocos numerosos.

O segundo padrão é o mais revelador. O grau ponderado é dominado por deputados de *dois partidos pequenos e coesos*, que coassinam intensamente entre si. Isso evidencia que a centralidade de coautoria mede *intensidade de articulação*, e *não poder institucional*. Retomando a definição operacional adotada (Seção 2.2.4), influência estrutural é a posição na rede, não o controle de relatorias, da presidência de comissões ou da pauta — mecanismos que se exercem por fora da coautoria. É por isso que lideranças formais e presidentes de comissão *não* aparecem no topo das métricas: sua influência opera por canais que a rede de coautoria, por construção, não captura. A rede revela quem *articula muito*, não necessariamente quem *decide muito*. Responde-se, assim, à terceira pergunta norteadora: a centralidade de coautoria *não* prediz influência política no sentido institucional — ela mede articulação e colaboração, dimensão estruturalmente distinta do poder decisório.

A concentração da articulação em poucos parlamentares — primeira pergunta norteadora — é quantificada pelo coeficiente de Gini (1921) da distribuição de cada centralidade (0 indica igualdade total; 1, concentração máxima) e pela participação dos dez deputados mais centrais. A Tabela 5 apresenta os valores de 2025.

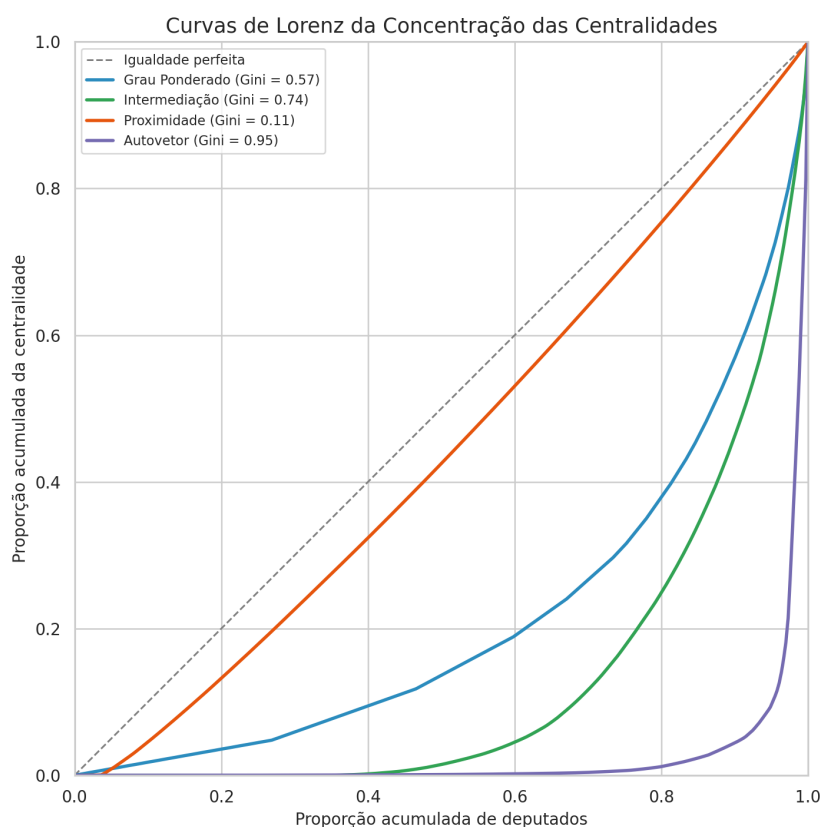
¹ Por se tratar de período eleitoral, os nomes de parlamentares e as siglas partidárias são apresentados de forma anonimizada (rótulos genéricos, locais a cada tabela) ao longo deste capítulo. A análise é estritamente estrutural e independe da identidade das legendas; a correspondência real é recuperável a partir dos dados abertos processados pelo artefato.

Tabela 5 – Concentração da centralidade em 2025 (coeficiente de Gini e participação dos 10 mais centrais).

Métrica	Gini	Top-10
Autovetor	0,951	82%
Intermediação	0,740	26%
Grau ponderado	0,566	21%
Proximidade	0,113	4%

Fonte: Elaborado pelo autor.

Figura 11 – Curvas de Lorenz da concentração das centralidades em 2025: quanto mais a curva se afasta da diagonal de igualdade, maior a concentração da métrica em poucos parlamentares.



Fonte: Elaborado pelo autor.

Os valores respondem afirmativamente à primeira pergunta norteadora: há forte concentração da articulação em poucos parlamentares, com intensidade que *varia* conforme a dimensão medida. A concentração é *extrema no autovetor* (Gini 0,951; os dez deputados mais centrais concentram 82% da métrica) — reflexo direto das bancadas coesas que coassinam quase exclusivamente entre si, formando câmaras de eco; *alta na intermediação* (0,740), pois o papel de ponte é raro; e *moderada no grau ponderado* (0,566). A proximidade, por sua vez, é quase igualitária (0,113) — comportamento esperado da métrica, cujos valores são naturalmente comprimidos

em redes conexas, nas quais todo vértice está a poucas arestas dos demais. Cabe a ressalva já discutida: trata-se de concentração da *articulação* (coautoria), não de poder institucional. O padrão é estável ao longo do período (o Gini do autovetor permanece entre 0,87 e 0,98 nos quatro anos). Por fim, a própria divergência entre os coeficientes (0,11, 0,57, 0,74 e 0,95) confirma que as quatro métricas de centralidade capturam dimensões distintas de influência — não são redundantes —, o que justifica reportá-las em conjunto. A Figura 11 representa graficamente essa concentração por meio das curvas de Lorenz das quatro centralidades: quanto mais distante da diagonal de igualdade, mais concentrada a métrica.

5.3 DETECÇÃO DE COMUNIDADES E TESTE DA HIPÓTESE CENTRAL

Esta seção examina a organização da rede em comunidades e submete a hipótese central (H1) ao teste estatístico. Avalia-se primeiro a robustez das partições, confrontando os dois algoritmos de detecção; em seguida, testa-se a significância da estrutura comunitária contra o modelo nulo ponderado e verifica-se sua correspondência com a filiação partidária.

5.3.1 Comparação Louvain vs. Label Propagation

A concordância entre as duas partições, medida pelo *Adjusted Rand Index* (ARI), é apresentada na Tabela 6.

Tabela 6 – Concordância entre Louvain e Label Propagation (ARI) por ano.

	2022	2023	2024	2025
Comunidades (Louvain)	15	12	14	16
Comunidades (Label Prop.)	19	13	12	10
ARI	0,325	0,281	0,416	0,423

Fonte: Elaborado pelo autor.

A concordância é *moderada* (ARI entre 0,281 e 0,423). Esse valor não deve ser lido como fragilidade, mas como o que de fato é: os dois algoritmos *convergem no essencial* — ambos detectam forte estrutura modular na mesma rede —, divergindo apenas na *granularidade* das fronteiras, isto é, em como partem os deputados situados entre blocos e em quantas comunidades reconhecem (o Label Propagation, reconhecidamente mais instável, varia de 10 a 19 comunidades). O ARI funciona, assim, como uma *verificação de consistência do sinal*: se os dois métodos concordassem apenas ao acaso ($ARI \approx 0$), a estrutura detectada seria suspeita; a concordância moderada e positiva indica um sinal estrutural real, ainda que suas bordas dependam do algoritmo. Não se trata, portanto, de exigir partições idênticas, mas de confirmar que a fragmentação em comunidades é robusta à escolha do método.

5.3.2 Análise pela Modularidade (Q) — Teste da Hipótese H1

A Tabela 7 apresenta o teste de significância da estrutura comunitária contra o modelo nulo ponderado.

Tabela 7 – Teste de significância da estrutura comunitária (modelo nulo ponderado, 200 permutações).

	2022	2023	2024	2025
Q_{obs}	0,596	0,640	0,634	0,634
Q_{nulo} (média)	0,370	0,420	0,415	0,467
p -valor	<0,005	<0,005	<0,005	<0,005
Significativo ($\alpha = 0,05$)	Sim	Sim	Sim	Sim

Fonte: Elaborado pelo autor.

Em todos os anos, a modularidade observada supera a média do modelo nulo, e *nenhum dos 200 grafos nulos atinge Q_{obs} ($p < 0,005$)*. A hipótese H1 é, portanto, corroborada: a estrutura de comunidades da rede é não-aleatória. Ressalte-se que o modelo nulo é *ponderado* — os pesos originais são reatribuídos às arestas reconfiguradas —, de modo que a comparação é homogênea com a modularidade observada, também ponderada. Corroborada H1, cria-se a *condição de validade* que autoriza a etapa seguinte: interpretar as comunidades detectadas como estruturas reais, e não como ruído. É sobre essa base que se responde à segunda pergunta norteadora.

A correspondência entre as comunidades detectadas e a filiação partidária foi avaliada de forma estritamente estrutural, sem recurso a qualquer classificação ideológica. A Tabela 8 apresenta a composição das principais comunidades de 2025.

Tabela 8 – Composição partidária das principais comunidades (Louvain, 2025; partidos anonimizados).

Comunidade	Deputados	Partido dominante (%) + aliados
1	118	Partido A (54%) + B, C, D
3	63	Partido E (59%) + F
9	47	Partido G (30%) + H, E
4	33	Partido I (39%) + E, J
6	24	Partido D (25%) + C, K
7	6	Partido L (67%)

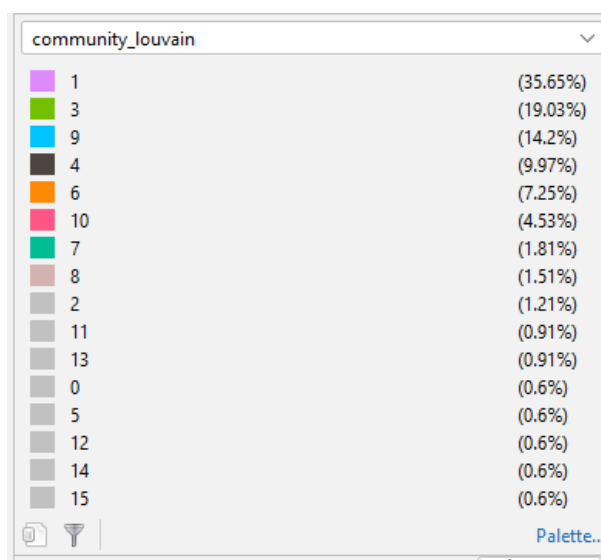
Fonte: Elaborado pelo autor.

O algoritmo *recupera a organização partidária* sem jamais receber rótulo de partido: cada comunidade é dominada por uma legenda real. Contudo, a correspondência é *parcial, e não idêntica* — medida pelo *Adjusted Rand Index* entre a partição de comunidades e a partição por filiação, obtém-se $ARI \approx 0,30$ e pureza partidária $\approx 48\%$ (proporção média de deputados no partido dominante de sua comunidade). Isso significa que as comunidades são *mais amplas*

que partidos isolados: enquanto algumas correspondem a um partido coeso (a comunidade 3, essencialmente de uma única legenda), outras agregam um partido dominante e legendas estruturalmente aliadas por coautoria (a comunidade 1 reúne um partido dominante e três legendas aliadas). Respondendo à pergunta norteadora, as comunidades correspondem, portanto, *mais a agrupamentos do tipo coalizão do que a linhas partidárias puras*. Essa leitura permanece no plano estrutural: descreve-se *quais partidos se agrupam por coautoria* — um fato observável —, sem atribuir-lhes rótulos de posição ideológica. O *Adjusted Rand Index* e a pureza aqui reportados são calculados de forma reprodutível por `scripts/compare_years.py`, que cruza a partição de comunidades (Louvain) com a filiação partidária a partir das métricas por deputado exportadas pelo *pipeline*.

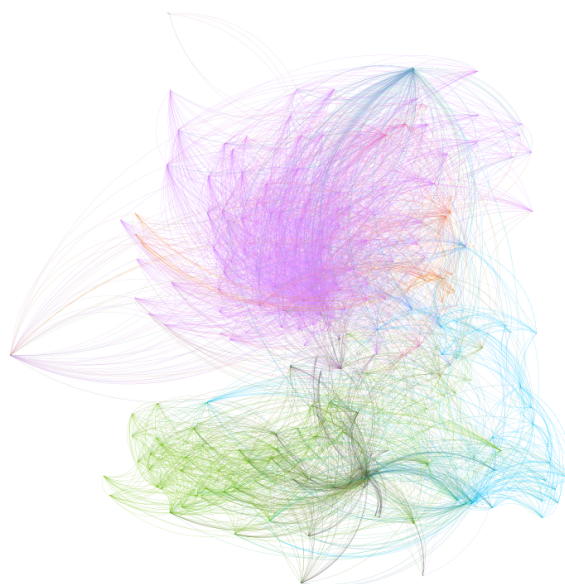
A Figura 13 oferece a validação visual dessa correspondência. As duas imagens usam *o mesmo posicionamento* de nós, gerado pelo algoritmo ForceAtlas 2 a partir exclusivamente das arestas (sem qualquer informação de partido ou comunidade). Na imagem superior (a), a coloração é por comunidade (Louvain); na inferior (b), pela filiação partidária. A concordância espacial entre as duas colorações confirma, de forma independente do algoritmo de detecção, que os agrupamentos são reais: onde a comunidade é um partido coeso, as cores coincidem; onde é uma coalizão, um único bloco espacial aparece multicolorido no painel de partidos — exatamente o comportamento previsto pelo ARI moderado.

Figura 12 – Legenda de cores por comunidade (Louvain) da Figura 13. A legenda de cores por partido foi omitida em razão do período eleitoral.

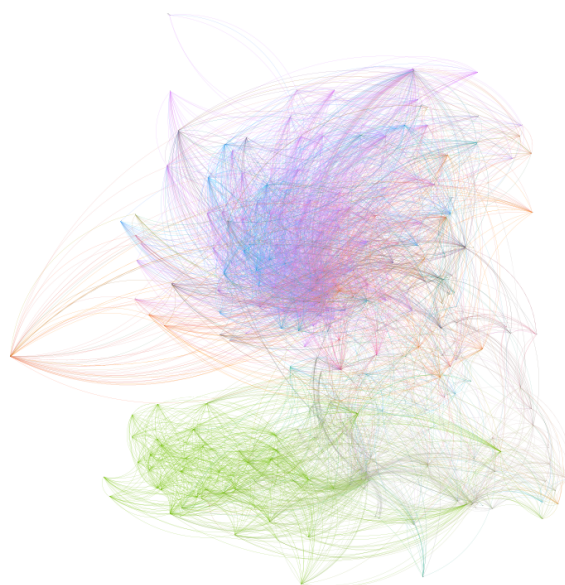


Fonte: Elaborado pelo autor com o Gephi.

Figura 13 – Rede de coautoria de 2025 (mesmo *layout* ForceAtlas 2), colorida por comunidade detectada (a) e por filiação partidária (b). Arestas com peso baixo omitidas para legibilidade.



(a) Cor por comunidade (Louvain)



(b) Cor por partido

Fonte: Elaborado pelo autor com o Gephi.

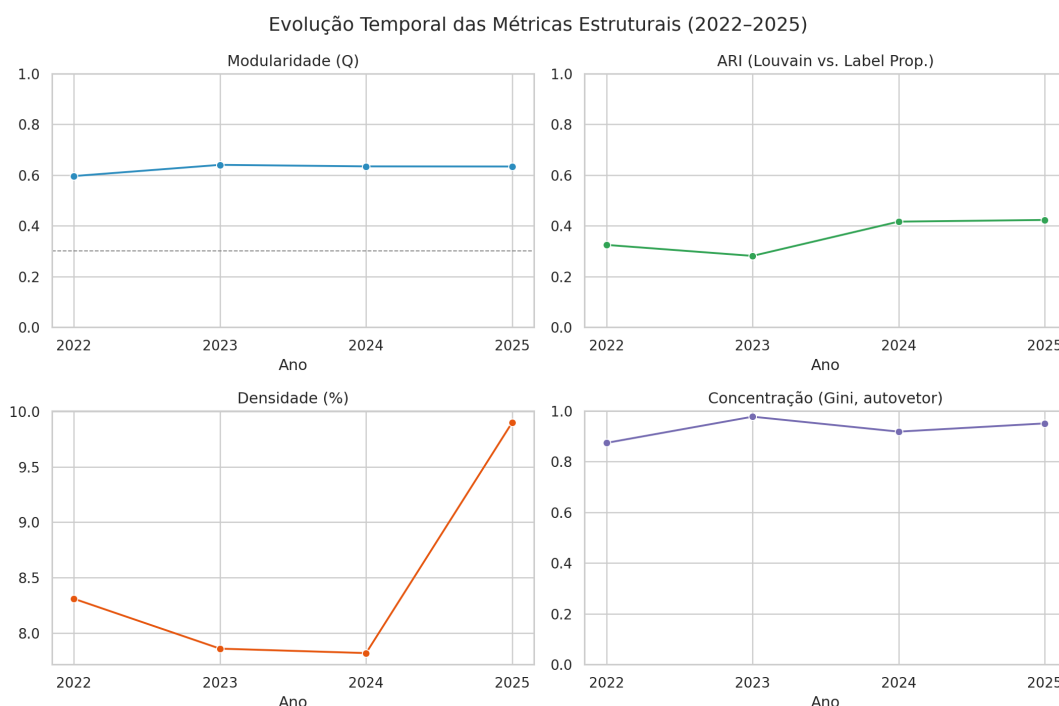
5.4 ANÁLISE TEMPORAL (2022–2025)

Tendo caracterizado a rede em detalhe para 2025, esta seção avalia se o padrão observado se sustenta ao longo dos quatro anos analisados. Discute-se primeiro a estabilidade das métricas estruturais e, na sequência, a leitura longitudinal do período, com as ressalvas metodológicas que o recorte impõe.

5.4.1 Estabilidade do Padrão Estrutural

A caracterização detalhada foi apresentada para 2025, ano mais recente e representativo do regime estável da 57ª Legislatura. Esta seção demonstra que o padrão descrito *se mantém em todo o período*. A modularidade observada permanece em uma faixa estreita, $Q_{\text{obs}} \in [0,596; 0,640]$, e H1 é corroborada nos quatro anos ($p < 0,005$; Tabela 7). A fragmentação em comunidades é, portanto, uma propriedade *persistente* do legislativo no período, e não um artefato de um ano específico. A Figura 14 sintetiza a evolução das principais métricas estruturais.

Figura 14 – Evolução das métricas estruturais (2022–2025): modularidade, ARI, densidade e concentração (Gini do autovetor).



Fonte: Elaborado pelo autor.

Observa-se estabilidade nas quatro dimensões: a modularidade oscila pouco em torno de 0,62; a concentração no autovetor mantém-se muito alta ($\text{Gini} > 0,87$); e a densidade sobe ligeiramente em 2025 (9,9%), acompanhando o maior número de coautorias registradas naquele ano.

5.4.2 Discussão Longitudinal

Uma ressalva metodológica é necessária. O ano de 2022 pertence à 56^a *Legislatura* — em seu encerramento e em ano eleitoral —, enquanto 2023 a 2025 compõem a 57^a. Assim, 2022 deve ser lido como ponto de referência para avaliar a *persistência* do padrão através de uma transição de legislatura, e não como termo de uma comparação ponto-a-ponto homogênea; a leve variação em número de nós e arestas entre os anos reflete, em parte, essa descontinuidade institucional. Ainda assim, o achado central sobrevive à ressalva: a estrutura modular significativa se mantém *apesar* da mudança de legislatura, o que sugere que a organização da colaboração legislativa em blocos é uma característica estável do funcionamento da Câmara, e não um efeito conjuntural. Essa leitura é deliberadamente prudente: a coautoria é um *proxy* de articulação, e o que se afirma limita-se ao que essa evidência estrutural autoriza — a persistência de uma organização em comunidades —, sem extrapolar para conclusões sobre coesão de voto ou disputa ideológica, que exigiriam outras fontes de dados.

6 CONCLUSÃO

Este capítulo encerra o trabalho. Retoma-se o problema de pesquisa e sintetizam-se os achados; sistematizam-se, então, as contribuições técnica e analítica do artefato; e, por fim, reconhecem-se com honestidade as limitações do estudo e apontam-se caminhos para trabalhos futuros.

6.1 SÍNTESE DOS ACHADOS

O objetivo geral deste trabalho — projetar, implementar e validar uma arquitetura de software reprodutível para análise de redes parlamentares — foi cumprido. Construiu-se um artefato modular e auditável que transforma dados legislativos brutos da Câmara dos Deputados em evidências analíticas mensuráveis, e sua eficácia foi demonstrada sobre o período 2022–2025. A hipótese central (H1) foi *corroborada nos quatro anos* ($p < 0,005$): a estrutura de comunidades da rede de coautoria é estatisticamente não-aleatória, propriedade que se mostrou persistente. Sobre essa base, as centralidades e as comunidades caracterizaram as *estruturas de influência* em seus dois níveis — os blocos (comunidades, correspondentes a coalizões partidárias) e os atores centrais (pontes e polos de articulação). O achado analítico mais relevante é a *distinção empírica entre articulação e poder institucional*: a centralidade de coautoria mede a intensidade da colaboração, não o controle das decisões, e essa fronteira foi tornada explícita pelos dados.

6.2 CONTRIBUIÇÕES

As contribuições do trabalho organizam-se em duas frentes complementares, coerentes com o paradigma DSR: a contribuição *técnica* — o artefato em si — e a contribuição *analítica* — o que sua aplicação ao período 2022–2025 revelou.

6.2.1 Contribuição técnica

A contribuição central e primária do trabalho, coerente com o paradigma DSR, é o *próprio artefato*: uma arquitetura de software modular, reprodutível e auditável para análise de redes parlamentares. Seguindo os princípios de Arquitetura Limpa e SOLID, o *pipeline* isola rigorosamente o núcleo analítico das preocupações de entrada e saída, é coberto por 217 testes automatizados — incluindo testes de integridade sobre os próprios *datasets* gerados — e é empacotado em Docker, garantindo execução idêntica em qualquer ambiente. O desacoplamento é reforçado por interfaces (contratos) na camada de persistência, que aplicam a inversão de dependência e tornam o artefato extensível a novos formatos sem alterar o núcleo; e sua correção é reimposta automaticamente por integração contínua a cada alteração do código. Trata-se de

uma *ferramenta reutilizável* e independente da fonte, disponibilizada em código aberto, apta a ser estendida para outras legislaturas, outros recortes ou outras casas legislativas.

6.2.2 Contribuição analítica

Como contribuição secundária — a demonstração de que o artefato funciona —, o trabalho oferece uma caracterização estrutural do legislativo brasileiro no período 2022–2025: (i) a existência de estrutura modular não-aleatória e persistente; (ii) comunidades que recuperam a organização partidária, ainda que no nível de coalizão e não de partido isolado; (iii) forte concentração da articulação em poucos parlamentares, especialmente nas bancadas coesas; e (iv) a já mencionada distinção entre a articulação capturada pela coautoria e o poder institucional formal. Essas descrições ilustram o tipo de evidência que a arquitetura é capaz de produzir de forma reproduzível.

6.3 LIMITAÇÕES

O reconhecimento explícito das limitações é parte da honestidade científica que o trabalho busca manter. As principais são:

- **Coautoria como *proxy*:** a rede mede articulação, não coesão de voto nem posição ideológica formal; dois adversários podem coassinar um projeto pontual, e aliados podem divergir em votações.
- **Centralidade $\neq$ poder institucional:** a rede não captura relatorias, presidência de comissões ou controle de pauta — canais de poder que operam por fora da coautoria.
- **Correspondência parcial com partidos:** as comunidades recuperam a organização partidária apenas no nível de coalizão ($ARI \approx 0,30$), não de partido isolado.
- **ARI moderado** (0,28–0,42) entre Louvain e Label Propagation: os algoritmos convergem no essencial, mas divergem na granularidade das fronteiras.
- **Recorte temporal:** 2022 pertence à 56ª Legislatura (ano eleitoral atípico), o que limita a comparação ponto-a-ponto com os demais anos.
- **Escopo dos filtros:** embora a modularidade tenha se mostrado robusta ao limiar `max_authors` (Seção 3.3.2), o filtro exclui integralmente as PECs da rede analisada; a exclusão de um tipo inteiro de proposição é uma decisão de escopo, adequada à definição de articulação adotada, mas que restringe o alcance da análise.
- **Acoplamento à fonte:** o artefato depende do esquema de arquivos CSV do Portal de Dados Abertos da Câmara; mudanças nesse esquema exigiriam adaptação da camada de extração.

6.4 TRABALHOS FUTUROS

- Análise de sensibilidade do filtro de tipos de proposição — em especial o impacto da inclusão das emendas (EMC) na estrutura —, complementando a análise do limiar `max_authors` já realizada.
- Comparação com o algoritmo de Leiden (Traag; Waltman; Eck, 2019), que garante comunidades bem conectadas.
- Cruzamento com dados de votação nominal, testando se a coautoria prediz votos.
- Extensão da série histórica e aprofundamento da análise longitudinal.
- Desenvolvimento de uma visualização interativa da rede — por exemplo, um painel web navegável — que permita explorar comunidades, centralidades e a evolução temporal de forma dinâmica, complementando as figuras estáticas geradas pelo artefato e a inspeção no Gephi.
- Aplicação de Redes Neurais de Grafos (GNNs) para predição de comportamento legislativo.
- Análise textual do conteúdo legislativo: os metadados brutos já extraídos incluem a ementa de cada proposição, permitindo, em trabalho futuro, técnicas de Processamento de Linguagem Natural (classificação temática, extração de tópicos) para investigar se comunidades e deputados centrais se concentram em pautas específicas.

6.5 CONSIDERAÇÕES FINAIS

Ao unir engenharia de software contemporânea e método científico, este trabalho demonstrou que é possível transformar a abundância de dados legislativos abertos em evidências estruturais mensuráveis, reproduzíveis e auditáveis. Mais do que os achados específicos sobre o período 2022–2025, o resultado duradouro é a *ferramenta* construída: um instrumento de código aberto que aproxima a análise de redes da realidade da Câmara dos Deputados e que fica à disposição para auditoria, extensão e reuso. Em um contexto que demanda transparência e escrutínio do processo legislativo, disponibilizar não apenas conclusões, mas o próprio meio reproduzível de chegar a elas, é a contribuição que se espera perdurar.

REFERÊNCIAS

- BARABÁSI, A.-L. **Network Science**. Cambridge University Press, 2016. Disponível em: <https://networksciencebook.com>. Acesso em: 20 ago. 2026.
- BLONDEL, V. D. et al. Fast unfolding of communities in large networks. **Journal of Statistical Mechanics: Theory and Experiment**, v. 2008, n. 10, p. P10008, 2008.
- BONACICH, P. Factoring and weighting approaches to status scores and clique identification. **Journal of Mathematical Sociology**, v. 2, n. 1, p. 113–120, 1972.
- BONACICH, P. Power and centrality: A family of measures. **American Journal of Sociology**, v. 92, n. 5, p. 1170–1182, 1987.
- BRANDES, U. A faster algorithm for betweenness centrality. **Journal of Mathematical Sociology**, v. 25, n. 2, p. 163–177, 2001.
- CORMEN, T. H. et al. **Introduction to Algorithms**. 3. ed. [S.l.]: MIT Press, 2009.
- DIESTEL, R. **Graph Theory**. 5. ed. [S.l.]: Springer, 2017.
- FIGUEIREDO, A. C.; LIMONGI, F. Bases institucionais do presidencialismo de coalizão. **Lua Nova: Revista de Cultura e Política**, n. 44, p. 81–106, 1998.
- FOWLER, J. H. Connecting the congress: A study of cosponsorship networks. **Political Analysis**, v. 14, n. 4, p. 456–487, 2006.
- FREEMAN, L. C. A set of measures of centrality based on betweenness. **Sociometry**, v. 40, n. 1, p. 35–41, 1977.
- GINI, C. Measurement of inequality of incomes. **The Economic Journal**, v. 31, n. 121, p. 124–126, 1921.
- HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. Exploring network structure, dynamics, and function using networkx. In: VAROQUAUX, G.; VAUGHT, T.; MILLMAN, J. (Ed.). **Proceedings of the 7th Python in Science Conference (SciPy2008)**. Pasadena, CA USA: [s.n.], 2008. p. 11–15.
- HEVNER, A. R. A three cycle view of design science research. **Scandinavian Journal of Information Systems**, v. 19, n. 2, p. 87–92, 2007.
- HEVNER, A. R. et al. Design science in information systems research. **MIS Quarterly**, v. 28, n. 1, p. 75–105, 2004.
- HUBERT, L.; ARABIE, P. Comparing partitions. **Journal of Classification**, v. 2, n. 1, p. 193–218, 1985.
- LANCICHINETTI, A.; FORTUNATO, S.; RADICCHI, F. Benchmark graphs for testing community detection algorithms. **Physical Review E**, v. 78, n. 4, p. 046110, 2008.
- MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. 1. ed. [S.l.]: Prentice Hall, 2017.

NEWMAN, M. E. J. Scientific collaboration networks. i. network construction and fundamental results. **Physical Review E**, v. 64, n. 1, p. 016131, 2001.

NEWMAN, M. E. J. **Networks: An Introduction**. [S.l.]: Oxford University Press, 2010.

NEWMAN, M. E. J.; GIRVAN, M. Finding and evaluating community structure in networks. **Physical Review E**, v. 69, n. 2, p. 026113, 2004.

NORTH, B. V.; CURTIS, D.; SHAM, P. C. A note on the calculation of empirical p values from monte carlo procedures. **American Journal of Human Genetics**, Elsevier, v. 71, n. 2, p. 439–441, 2002.

RAGHAVAN, U. N.; ALBERT, R.; KUMARA, S. Near linear time algorithm to detect community structures in large-scale networks. **Physical Review E**, v. 76, n. 3, p. 036106, 2007.

TRAAG, V. A.; WALTMAN, L.; ECK, N. J. van. From louvain to leiden: Guaranteeing well-connected communities. **Scientific Reports**, v. 9, n. 1, p. 5233, 2019.

WIERINGA, R. J. **Design Science Methodology for Information Systems and Software Engineering**. [S.l.]: Springer, 2014.

ZUCCO, C. Ideology or what? legislative behavior in multiparty presidential settings. **The Journal of Politics**, v. 71, n. 3, p. 1076–1092, 2009.

APÊNDICE A – ESTRUTURA DO REPOSITÓRIO DE CÓDIGO

O código-fonte completo do artefato desenvolvido neste trabalho está disponível publicamente em <https://github.com/fvilhalva/parliament-graph-architecture>. O repositório segue a seguinte estrutura modular:

- `src/config/`: configuração centralizada (variáveis de ambiente, constantes, logger);
- `src/core/`: núcleo analítico (classe `ParliamentaryGraph` e submódulo `algorithms/`);
- `src/extraction/`: camada de extração (`ChamberExtractor`);
- `src/models/`: entidades de domínio (`Deputy`, `Proposition`, `CoauthorshipEdge`);
- `src/processing/`: limpeza, filtros e conversão para objetos de domínio (`ChamberProcessor`);
- `src/repository/`: persistência (CSV, GEXF, SQLite, JSON);
- `src/visualization/`: geração automática de figuras;
- `src/tests/`: suíte de testes (217 testes);
- `scripts/`: utilitários auxiliares (análise comparativa entre anos);
- `data/`: artefatos gerados em tempo de execução (não versionados);
- `Dockerfile`, `docker-compose.yml`, `requirements.txt`: empacotamento reprodutível.